

学习面向数据处理集群的调度算法

Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrisnan, Zili Meng*, Mohammad Alizadeh

MIT Computer Science and Artificial Intelligence Laboratory *Tsinghua University

{hongzi,malte,bjjvnt,alizadeh}@csail.mit.edu,mengz15@mails.tsinghua.edu.cn

摘要

要在分布式计算集群上高效调度数据处理作业，需要复杂的算法。现有系统采用简单、通用的启发式方法，忽略工作负载的特征，因为为每一种工作负载开发并调优专门的调度策略并不可行。本文表明，现代机器学习技术能够自动生成高效的调度策略。

Decima使用强化学习(RL)和神经网络，学习针对具体工作负载的调度算法；除了“最小化平均作业完成时间”这类高层目标之外，不需要任何人工指令。然而，现成的强化学习技术无法处理调度问题的复杂性和规模。为构建Decima，我们必须为作业依赖图开发新的表示方式，设计可扩展的强化学习模型，并提出能够处理连续随机作业到达的强化学习训练方法。

我们将原型集成到Spark，并在一个由25个节点组成的集群上开展实验。结果表明，相比人工调优的调度启发式方法，Decima至少缩短21%的平均作业完成时间；在集群负载较高的时段，性能提升最高达到2 $\times$ 。

CCS 分类: 软件及其工程 $\rightarrow$ 调度; 网络 $\rightarrow$ 网络资源分配; 计算方法 $\rightarrow$ 强化学习

关键词: 资源管理、作业调度、强化学习

ACM 参考文献格式:

Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrisnan, Zili Meng and Mohammad Alizadeh. 2019. 学习面向数据处理集群的调度算法. In SIGCOMM '19, August 19-23, 2019, Beijing, China. ACM, Beijing, China, 18 pages. <https://doi.org/10.1145/3341302.3342080>

1 引言

高效利用昂贵的计算集群对企业至关重要：在大规模部署中，即使利用率只有小幅提升，也能节省数百万美元[11, §1.2]。

集群调度器是实现这些节省的关键。好的调度策略会紧凑地放置工作，以减少资源碎片[34, 36, 76]；根据用户感知的延迟等高层指标确定作业优先级[77]；并避免低效的配置[28]。

当前集群调度器依赖启发式方法，优先追求通用性、易于理解和容易实现，而非在特定工作负载上达到理想性能。

现有系统使用公平调度[8, 31]、最短作业优先和简单装箱策略[34]等通用启发式方法，因而放弃了一些潜在的性能优化机会。

例如，广泛使用的调度器忽略了容易获得的作业结构信息(即，内部依赖关系)，也忽略了与作业输入规模相适应的高效并行度。

但利用这些信息、针对具体工作负载的调度策略，需要专家知识，也需要大量设计、实现和验证工作。

许多组织没有这种能力，或者投入并不划算，因为人工成本超过了潜在节省。

本文表明，现代机器学习技术能够通过自动学习针对具体工作负载的高效调度策略，帮助绕开这一权衡。

我们提出Decima¹，这是一个面向具有阶段依赖关系的数据处理作业的通用调度服务。

许多系统将作业阶段及其依赖关系编码为有向无环图(DAG) [10, 19, 42, 80]。高效调度DAG会涉及困难的算法问题，其最优解在计算上不可处理[36]。

只要给出一个高层目标(例如，最小化平均作业完成时间)，Decima就能利用已有监控信息和历史工作负载日志，自动学习复杂的调度策略。

例如，Decima会学习为不同作业分配不同的资源份额，以优化整体性能；也会学习各作业适宜的并行度，避免对内在并行性较低的作业投入过多资源而只获得递减的收益。

这些策略所需的算法和阈值取决于工作负载，现有做法需要繁琐的人工调度器定制。

Decima使用现代强化学习技术，通过经验学习调度策略。

强化学习适合学习调度策略，因为它能从实际工作负载和运行条件中学习，不必依赖不准确的假设。

Decima用神经网络编码调度策略，并通过大量仿真实验训练：在实验中调度工作负载，观察结果，逐步改进策略。

但Decima的贡献超出了把现成强化学习算法直接应用于调度：为了成功学习高质量策略，我们必须提出新的数据表示、调度动作表示和强化学习训练技术。

首先，集群调度器必须扩展到数百个作业和数千台机器，并且为每个作业在可能的数百种配置之间作出决策(例如，不同并行度)。与传统强化学习应用(例如游戏[61, 70]、机器人控制[51, 67])相比，这使问题规模大得多：调度器可用信息的数量(即状态空间)和必须考虑的候选选择数量(即动作空间)都更大。²

我们设计了一种可扩展的神经网络架构：它结合图神经网络[12, 23, 24, 46]和作出调度决策的策略网络，无需人工特征工程即可处理作业及集群信息。无论作业DAG的大小、形状以及作业或机器数量如何，网络都重复使用一小组基本操作来处理DAG并作出调度决策。这些操作是训练期间学习得到的参数化函数，并专门针对调度领域设计，例如，确保图神经网络可以表达DAG的关键路径等性质。相比对调度问题的朴素编码，这种网络设计大幅降低模型复杂度，是高效学习、快速训练和低延迟调度决策的关键。

其次，传统强化学习算法无法用持续流式到达的作业训练模型。

作业到达的随机性，可能使强化学习算法无法区分两次决策的观察结果之所以不同，究竟是到达模式不同，还是策略决策质量不同。

SIGCOMM '19, August 19-23, 2019, Beijing, China

© 2019 Association for Computing Machinery.

这是本工作的作者版本，发布于此仅供个人使用，不得再分发。正式出版版本刊载于 SIGCOMM '19, August 19-23, 2019, Beijing, China, <https://doi.org/10.1145/3341302.3342080>.

¹在罗马神话中，Decima 负责丈量生命之线并决定其命运。

²例如，围棋 [71] 的状态可用 $19 \times 19 = 361$ 个数表示，这个数也限制了每轮合法落子的数量。

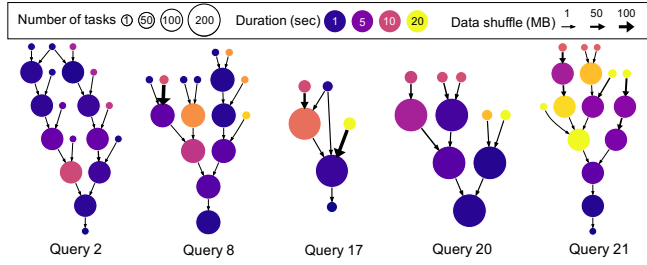


图 1: 数据并行作业具有复杂的数据流程图, 例如图中在 Spark 上运行的 TPC-H 查询; 各节点的任务数量、任务时长及输入/输出规模各不相同。

图中术语: Number of tasks 为任务数; Duration (sec) 为任务时长(秒); Data shuffle (MB) 为数据重分布量 (MB); Query 为查询。

此外, 强化学习策略在训练初期不可避免地会作出较差的决策。因此, 面对无界的输入作业流, 策略必然积累作业, 形成无法消除的积压。

在这种状态下花费大量训练时间探索动作, 并不能改进策略。为处理后一个问题, 我们在训练开始时提前结束训练“回合”, 再逐渐延长回合长度。

这样, 策略可以先学会处理简单、短的作业序列, 再逐步过渡到更困难的到达序列。

为应对作业到达的随机性, 我们使用近期针对随机输入环境的强化学习方法[55], 以实际经历的作业到达序列为条件, 构造训练反馈。

这会在反馈中分离出调度策略自身的贡献, 使学习能够处理随机作业到达的策略成为可能。

我们将 Decima 集成到 Spark, 在实验测试平台及阿里巴巴生产集群的工作负载轨迹上进行评估[6, 52]。³

评估表明, Decima 在一个由 25 个节点组成的 Spark 集群上优于现有启发式方法, 使 TPC-H 查询混合作业的平均完成时间至少缩短 21%。Decima 的策略在高负载时段尤其有效, 相比现有启发式方法, 作业完成时间最高改善 2x。

Decima 还可以扩展到 CPU 和内存的多资源调度; 相比 Graphene[36]等已有方案, 其平均作业完成时间缩短 32-43%。

本文的主要贡献如下:

- (1) 一种可扩展的神经网络设计, 可以处理任意形状和大小的 DAG, 调度 DAG 的阶段, 并为每个作业设定高效的并行度 (第 5.1-5.2 节)。
- (2) 一组强化学习训练技术, 首次使调度器能够通过训练处理无界的随机作业到达序列 (第 5.3 节)。
- (3) Decima 及其原型实现: 它是首个基于强化学习、能调度复杂数据处理作业且无需人工输入便可学习针对具体工作负载的调度策略的调度器 (第 6 节)。
- (4) 在仿真和真实 Spark 集群中对 Decima 的评估, 以及与当时先进调度启发式方法的比较 (第 7 节)。

2 研究动机

Hive、Pig、SparkSQL 和 DryadLINQ 等数据处理系统与查询编译器会创建具有 DAG 结构的作业, 其中各处理阶段通过输入/输出依赖连接起来 (图 1)。

生产集群中常见的重复作业[4], 通常可以获得相对合理的运行时间和中间数据规模估计。

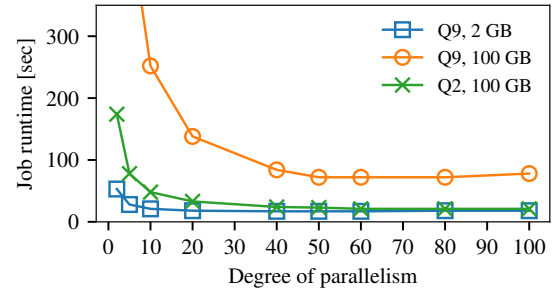


图 2: TPC-H 查询随并行度增加的扩展性不同: 处理 100 GB 输入的 Q9 在增加到 40 个并行任务之前仍能加速, Q2 在 20 个任务后便不再获益; 处理 2 GB 输入的 Q9 只需 5 个任务。在混合工作负载中为这些曲线选择“最佳区间”十分困难。

图中术语: Degree of parallelism 为并行度; Job runtime [sec] 为作业运行时间 (秒)。

但多数集群调度器在决策中忽略这种作业结构, 而依赖粗粒度公平共享[8, 16, 31, 32]、固定优先级[77], 以及人工指定各作业并行度[68, §5]等方法。

现有调度器基本忽略这些丰富且易于获取的作业结构信息, 因为设计能够利用它们的调度算法十分困难。

我们用两个具体例子说明, 在调度决策中利用作业专有信息面临的挑战: (i) 感知依赖关系的调度; (ii) 自动选择合适的并行任务数量。

2.1 感知依赖关系的任务调度

实际作业 DAG 通常包含数十或数百个阶段, 各阶段的时长与并行任务数量不同, 依赖结构也很复杂。

理想调度应尽可能让互不依赖的阶段并行运行, 并确保只要存在可用资源, 就不会有阶段因依赖关系而被阻塞。

为此, 调度器需要理解依赖结构并提前规划。

这一“DAG 调度问题”在算法上很困难, 例如, Grandl 等人的示例[36, §2.2], 以及我们在附录 A 详细描述示例。

理论研究[18, 20, 48, 69]主要关注该问题的简单实例, 无法涵盖真实数据处理集群的复杂性, 例如在线作业到达、多个 DAG、每阶段多个任务、不同作业的内在并行性差异, 以及在机器间迁移作业的开销等。比如, Agrawal 等人在近期论文[5]中证明, 在每个作业阶段只有一个任务的基本模型中, 两种简单 DAG 调度策略——最短作业优先和最后到达处理器共享——具有常数竞争比。但我们的结果表明 (第 2.3 节、第 7 节), 这些策略在真实 Spark 集群上远非最优。

因此, 为所有可能的 DAG 组合设计能够生成最优调度的算法, 在计算上不可处理[36, 57]。

现有调度器避开这一挑战: 阶段一旦可运行, 就把其任务入队, 或者按任意顺序运行各阶段。

2.2 设定合适的并行度

除理解依赖关系之外, 理想调度器还必须理解怎样在各作业间合理划分有限资源。

不同作业处理的数据量和可用并行工作量各不相同。

具有大量输入或中间数据的作业, 能够高效利用额外并行性; 相反, 输入数据少或操作不易高效并行化的作业, 在并行度略高之后就会出现收益递减。

图 2 展示了这一点: 两种在 Spark 上运行的 TPC-H 查询[73]获得更多资源、可以运行更多并行任务后, 其作业运行时间呈现不同变化。

³我们使用了阿里巴巴公开 cluster-trace-v2018 轨迹的一个较早版本。

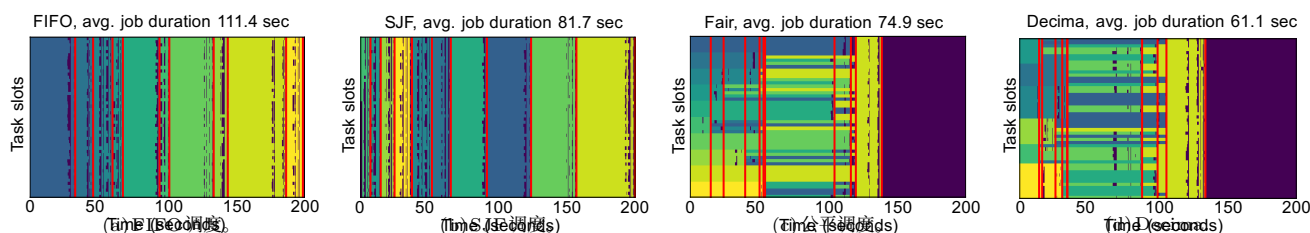


图3: 在具有50个任务槽位(执行器)的集群上, 相比Spark的FIFO调度器, Decima将10个随机TPC-H查询的平均作业完成时间(JCT)缩短45%; 相比公平调度器, 缩短19%。不同颜色表示不同查询, 红色竖线表示作业完成, 紫色表示空闲。

图中术语: Task slots为任务槽位; Time(seconds)为时间(秒); avg. job duration为平均作业时长; Fair为公平调度。

即使都处理100GB输入, Q2和Q9的扩展性仍然非常不同: Q9在并行任务数达到40之前仍显著加速, 而Q2在20个任务后只能获得很小收益。

但当Q9处理较小的2GB输入时, 所需并行任务不超过10个。

对所有作业而言, 在曲线的“最佳区间”之外继续增加并行任务, 带来的额外收益都会递减。

因此, 调度器应判断哪一个作业从额外资源中获得的边际收益最大, 并据此为每个作业选择最佳并行区间。

现有调度器大多绕开这个问题。多数调度器把选择并行任务数的负担交给用户[68, §5], 或者依赖一个独立的“自动扩容”组件, 其依据只是粗略的启发式规则[9, 28]。

实际上, 许多公平调度器[31, 43]划分资源时并不关心决策效率: 有时, “不公平”的调度反而产生更高效的整体执行。

2.3 Spark上的示例

上述两方面只是调度器利用工作负载知识的两个例子。

为达到最佳性能, 调度器还须考虑其他因素, 例如执行顺序(例如, 优先处理短作业), 以及避免资源碎片[34, 77]。

像Decima这样同时考虑所有这些维度, 会产生显著区别。

为说明这一点, 我们在一个具有50个并行任务槽位的Spark集群上, 运行10个随机选择的TPC-H查询[73], 其输入规模从长尾分布中抽取。⁴

图3可视化了以下策略生成的调度: (a) Spark默认的FIFO调度; (b) 严格优先处理短作业的最短作业优先(SJF)策略; (c) 更贴近实际、在作业间动态划分任务槽位的公平调度器; 以及(d) Decima学习得到的调度策略。

我们测量这10个作业的平均完成时间(JCT)。利用图结构使Decima相比朴素FIFO调度器将平均JCT缩短45%, 相比公平调度器缩短19%。

它通过快速完成短作业, 以及最大化并行处理效率来实现这种加速: 前40秒内便有5个作业完成。

SJF将所有任务槽位分配给剩余作业中最短的一个, 以尽早完成它, 但这样做并不高效; Decima则让各作业在接近最佳并行区间的位置运行。

通过控制并行度, Decima相比SJF将完成全部作业的总时间缩短30%。

此外, 与公平调度不同, Decima在作业间非均匀地划分任务槽位, 使平均JCT缩短19%。

要设计通用启发式方法来实现这些收益十分困难, 因为每增加一个维度(DAG结构、并行度、作业规模等), 都会增加复杂性并产生新的边界情况。

Decima提供了一种新选择: 利用数据驱动技术, 自动学习能够获得上述收益、针对具体工作负载的策略。

⁴工作负载及集群配置的细节见第7节。

Decima无需高层目标(例如, 最小化平均JCT)之外的人工指导, 也无需显式建模系统或工作负载。

3 Spark 中的 DAG 调度问题

Decima是一个通用框架, 用于学习具有DAG结构的作业的调度算法。

为使描述具体, 下文以Spark系统为背景介绍其设计。

Spark作业由一个DAG构成, 图中的节点是作业的执行阶段。每个阶段对应一种操作, 系统在该阶段输入数据的许多分片上并行执行它。

输入来自一个或多个父阶段的输出; 每个分片由单个任务处理。

一个阶段的所有父阶段完成后, 该阶段的任务便可运行。能够并行运行多少任务, 取决于该作业持有的执行器数量。

通常, 一个阶段的任务数多于执行器数, 因此这些任务分成多“波”执行。

Spark主节点根据用户请求分配执行器, 默认情况下执行器一直绑定在作业上, 直到该作业完成。

不过, Spark也支持根据待处理任务的等待时间动态分配执行器[9]; 但执行器在作业间迁移会产生一定开销, 例如销毁和启动JVM。

因此, Spark必须处理三类调度决策: (i) 决定每个作业获得多少执行器; (ii) 决定每个作业接下来运行哪些阶段的任务; (iii) 在执行器空闲时决定下一个运行哪个任务。

某个阶段完成后, 该作业的DAG调度器会激活依赖它的子阶段, 并将子阶段的任务放入较低层的任务调度器队列。

任务调度器维护任务队列, 每当执行器空闲时便从队列中分配一个任务。

我们允许调度器按需要在作业DAG之间迁移执行器, 即动态分配。

Decima关注DAG调度(即, 接下来运行哪个阶段)以及执行器分配(即, 各作业的并行度)。

由于同一阶段的任务运行相同代码并请求相同资源, 我们直接使用Spark已有的任务级调度。

4 概述与设计挑战

Decima将调度器表示为一个使用神经网络作出决策的智能体, 这个网络下文称为策略网络。在调度事件发生时, 例如阶段完成并释放执行器, 或作业到达并增加一个DAG, 智能体以当前集群状态为输入, 输出一个调度动作。总体而言, 状态记录调度器队列中各DAG及执行器的状态; 动作则决定任意时刻各执行器处理哪些DAG阶段。

Decima通过大量离线仿真实验, 用强化学习训练神经网络。在实验中, Decima尝试调度工作负载, 观察结果, 并在每次

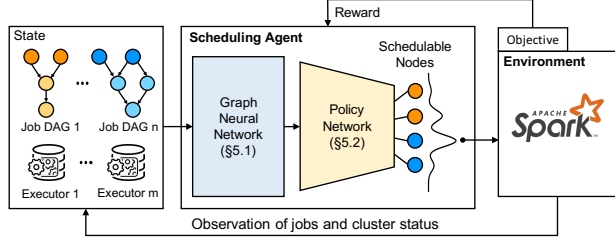


图4: 在 Decima 的强化学习框架中, 调度智能体观察集群状态, 决定作用于集群环境的调度动作, 并根据高层目标获得奖励。智能体利用图神经网络将作业 DAG 转换为向量, 供输出动作的策略网络使用。图中术语: State 为状态; Job DAG 为作业 DAG; Executor 为执行器; Scheduling Agent 为调度智能体; Graph Neural Network 为图神经网络; Policy Network 为策略网络; Scheduling Nodes 为可调度节点; Environment 为环境; Objective 为目标; Reward 为奖励; Observation of jobs and cluster status 为观察作业与集群状态。

动作后向智能体提供奖励。奖励由 Decima 的高层调度目标决定, 例如最小化平均 JCT。强化学习算法利用这一奖励信号逐步改进调度策略。附录 B 简要介绍强化学习基础。

Decima 的强化学习框架 (图4) 具有通用性, 能够应用于不同系统和目标。第5节介绍其在一组相同执行器上调度 DAG、最小化平均 JCT 的设计。第7节的结果展示如何用同一设计调度多种资源 (例如 CPU 和内存), 优化总完工时间 [65] 等其他目标, 并根据底层系统 (例如不同的跨机器作业迁移开销) 学习性质不同的策略。

挑战。 Decima 的设计解决三个关键挑战:

- (1) **可扩展的状态信息处理。** 调度器作出决策时必须考虑大量动态信息: 数百个各含数十个阶段的作业 DAG, 以及可能各处于不同状态 (例如, 分配给不同作业) 的执行器。通过神经网络处理全部信息很有挑战, 尤其是神经网络通常要求输入为固定大小的向量。
- (2) **巨大的调度决策空间。** 调度器必须将可能的数千个可运行阶段映射到可用执行器上。映射空间呈指数规模, 这给强化学习带来挑战, 因为它必须在训练中“探索”动作空间才能学习好的策略。
- (3) **针对持续随机作业到达的训练。** 必须训练调度器, 使其能够持续处理随机到达的作业。但在连续作业到达过程中训练并不容易, 因为强化学习算法通常需要具有有限时间范围的训练“回合”。此外, 我们发现到达过程的随机性会给奖励增加方差和噪声, 导致强化学习训练困难。

5 设计

本节按解决上述三个挑战的方式介绍 Decima 的设计: 可扩展地处理状态信息 (第5.1节); 将调度决策高效编码为动作 (第5.2节); 以及在持续随机作业到达条件下进行强化学习训练 (第5.3节)。

5.1 可扩展的状态信息处理

每次观察状态时, Decima 都须将作业 DAG 和执行器状态转换为特征, 传给策略网络。一种选择是创建包含全部状态信息的扁平特征向量, 但这种方式无法扩展到任意数量、任意大小和形状的 DAG。即使严格限制作业数和阶段数, 处理高维特征向量仍需要一个很大的策略网络, 而这种网络难以训练。

Decima 使用图神经网络实现扩展性, 将状态信息 (例如, 作业阶段的属性、DAG 依赖结构等) 编码或“嵌入”到一组嵌入向量中。我们的方法基于图卷积神经网络 [12, 23, 46], 但针对调度作了定制。表1定义所用记号。

对象	符号	对象	符号
作业	i	节点特征向量	x_v^i
阶段 (DAG 节点)	v	节点嵌入	e_v^i
节点 v 的子节点	$\xi(v)$	作业嵌入	y^i
作业 i 的 DAG	G_i	全局嵌入	z
作业 i 的并行度	l_i	节点得分	q_v^i
非线性函数	f, g, q, w	并行度得分	w_l^i

表1: 第5节所用记号。

图嵌入以作业 DAG 为输入, 节点携带一组阶段属性 (例如, 剩余任务数、预期任务时长等), 并输出以下三种嵌入:

- (1) 节点嵌入, 记录该节点及其子节点的信息, 例如从该节点开始的关键路径上的聚合作量;
- (2) 作业嵌入, 聚合整个作业 DAG 的信息, 例如作业总工作量;
- (3) 全局嵌入, 将所有作业嵌入的信息组合成集群级摘要, 例如作业数量和集群负载。

这些嵌入究竟保存什么信息并非硬编码。Decima 通过端到端训练, 自动学习什么信息在统计上重要, 以及怎样从输入 DAG 中计算这些信息。

换言之, 嵌入可以视为图神经网络无需人工特征工程就能学会计算的特征向量。

Decima 的图神经网络具有扩展性, 因为它重复使用一组公共操作作为基本构件, 计算上述嵌入。这些基本构件本身由小型神经网络实现, 作用于相对低维的输入向量。

节点嵌入。 给定 DAG G_i 中各节点对应的阶段属性向量 x_v^i , Decima 构建节点嵌入 $(G_i, x_v^i) \mapsto e_v^i$ 。

所得 e_v^i 是一个向量, 例如位于 $\mathbb{R}^{16}$ 中, 记录了从 v 可达的所有节点的信息 (即, v 的子节点、子节点的子节点, 依此类推)。

为计算这些向量, Decima 从 DAG 叶节点开始, 通过一系列消息传递步骤, 将信息由子节点传播至父节点 (图5a)。

在每个消息传递步骤中, 当节点 v 的子节点已经聚合了它们所有子节点的消息 (图5a 示例中的着色节点) 后, 该节点按如下方式计算自身嵌入:

$$e_v^i = g \left[\sum_{u \in \xi(v)} f(e_u^i) \right] + x_v^i, \quad (1)$$

其中, $f(\cdot)$ 和 $g(\cdot)$ 是作用于输入向量的非线性变换, 由小型神经网络实现; $\xi(v)$ 表示 v 的子节点集合。

第一项是一种通用非线性聚合操作, 用于汇总 v 的子节点嵌入; 将这一汇总项加到 v 的特征向量 (x_v) 上, 就得到 v 的嵌入。

Decima 在所有节点、所有消息传递步骤中复用相同的非线性变换 $f(\cdot)$ 和 $g(\cdot)$ 。

多数已有图神经网络架构 [23, 24, 46] 使用形如 $e_v = \sum_{u \in \xi(v)} f(e_u)$ 的聚合操作来计算节点嵌入。但我们发现, 在式 (1) 中增加第二个非线性变换 $g(\cdot)$, 对于学习强大的调度策略非常关键。

原因是: 没有 $g(\cdot)$, 图神经网络就无法计算某些有用的调度特征。例如, 它无法计算 DAG 的关键路径 [44], 因为这一计算要求消息传递时对子节点取最大值。⁵ 结合两个非线性变换 $f(\cdot)$ 和 $g(\cdot)$, 使 Decima 能表达多种聚合函数。例如, 若 f 和 g 都为恒等变换, 则聚合操作对子节点嵌入求和; 若 $f \sim \log(\cdot/n)$,

⁵ 从节点 v 出发的关键路径可按如下方式计算: $\text{cp}(v) = \max_{u \in \xi(v)} \text{cp}(u) + \text{work}(v)$, 其中 $\text{work}(\cdot)$ 表示节点 v 上的总工作量。

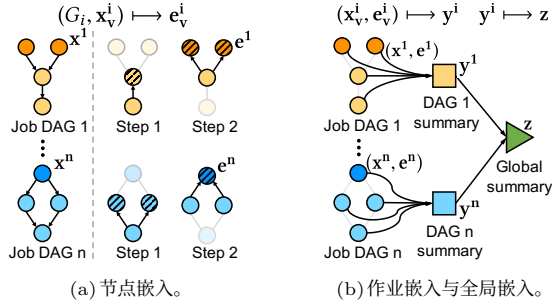


图 5: 图神经网络将各 DAG 节点的原始信息转换为向量表示。该例展示了两步局部消息传递和两层汇总。

图中术语: Job DAG 为作业 DAG; Step 为步骤; DAG summary 为 DAG 汇总; Global summary 为全局汇总。

$g \sim \exp(n \times \cdot)$, 且 $n \rightarrow \infty$, 则聚合操作计算子节点嵌入的最大值。⁶附录 E 给出关于这种嵌入的实证研究。

作业嵌入与全局嵌入。图神经网络还为每个 DAG G_i 计算全部节点嵌入的摘要: $\{(x_v^i, e_v^i), v \in G_i\} \mapsto y^i$; 并计算跨全部 DAG 的全局摘要:

$\{y^1, y^2, \dots\} \mapsto z$. 为计算这些嵌入, Decima 为每个 DAG 添加一个汇总节点, 将该 DAG 的全部节点作为它的子节点 (图 5b 中的方形)。这些 DAG 级汇总节点又作为一个全局汇总节点的子节点 (图 5b 中的三角形)。

这些汇总节点的嵌入也用式 (1) 计算。每一层汇总都具有独立的非线性变换 f 和 g ; 换言之, 图神经网络总共使用六个非线性变换, 每层汇总使用两个。

5.2 将调度决策编码为动作

编码调度决策的关键挑战, 在于处理大动作空间的学习复杂度和计算复杂度。

考虑一种朴素做法: 给定第 5.1 节的嵌入, 一次返回将全部执行器分配给作业阶段的方案。它必须从指数规模的组合中选择动作。另一个极端是, 每当一个执行器可用, 就调用调度智能体选择一个阶段。其动作空间小得多, 为 $O(\text{阶段数量})$, 但为一组给定作业完成调度需要很长的动作序列。在强化学习中, 较大的动作空间和较长的动作序列都会增加样本复杂度并减慢训练 [7, 72]。

Decima 将调度决策分解成一系列二维动作, 平衡动作空间大小与所需动作数。每个动作输出: (i) 接下来安排的阶段; (ii) 该阶段所属作业可使用的执行器数量上限。

调度事件。当任意作业 DAG 中的可运行阶段集合发生变化时, Decima 调用调度智能体。可运行阶段是指所有父阶段已完成, 且至少有一个任务正在等待的阶段。

调度事件在以下情况下发生: (i) 一个阶段的任务已全部配, 不再需要更多执行器; (ii) 一个阶段完成, 使一个或多个子阶段的任务可运行; (iii) 新作业到达系统。

在每次调度事件中, 智能体通过一个或多个动作安排一组空闲执行器。具体而言, 它将第 5.1 节的嵌入向量传入策略网络, 输出二维动作 (v, l_i) , 包含阶段 v 及其所属作业的并行度上限 l_i 。这里, 阶段 v 属于作业 i 。若作业 i 当前拥有的执行器少于 l_i , 则 Decima 向 v 分配执行器, 直到达到这一上限。若在该调度动作之后仍有空闲执行器, Decima 再次调用智能体, 选

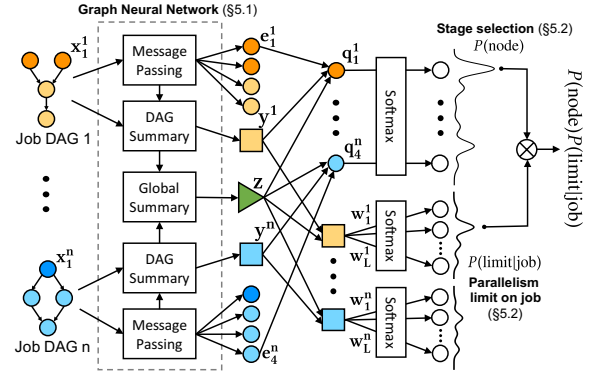


图 6: 对作业 i 的每个节点 v , 策略网络使用节点嵌入 e_v^i 、作业嵌入 y^i 和全局嵌入 z , 计算: (i) 用于采样待调度节点的得分 q_v^i ; (ii) 用于采样该节点所属作业并行度上限的得分 w_l^i 。

图中术语: Message Passing 为消息传递; DAG Summary 为 DAG 汇总; Global Summary 为全局汇总; Graph Neural Network 为图神经网络; Stage selection 为阶段选择; Parallelism limit on job 为作业并行度上限; Softmax 保留算法名称; node, limit, job 分别表示节点、上限和作业。

择另一个阶段和并行度上限。这一过程重复进行, 直到全部执行器已分配, 或不存在可运行阶段。Decima 要求并行度上限 l_i 大于当前分配给作业 i 的执行器数, 保证每个动作至少安排一个新执行器, 从而使该过程在有限步内结束。

阶段选择。图 6 展示 Decima 的策略网络。对于时刻 t 发生的调度事件, 设状态为 s_t , 策略网络按如下方式选择待调度阶段: 对作业 i 中的节点 v , 计算得分 $q_v^i \triangleq q(e_v^i, y^i, z)$; 其中 $q(\cdot)$ 是一个将嵌入向量映射为标量的评分函数。这些向量由第 5.1 节的图神经网络输出。与嵌入步骤类似, 评分函数也是由神经网络实现的非线性变换。得分 q_v^i 表示调度节点 v 的优先级。随后, Decima 用 softmax 操作 [17], 根据优先级得分计算选择节点 v 的概率:

$$P(\text{node}=v) = \frac{\exp(q_v^i)}{\sum_{u \in \mathcal{A}_t} \exp(q_u^{j(u)})}, \quad (2)$$

其中, $j(u)$ 是节点 u 所属的作业; $\mathcal{A}_t$ 是时刻 t 可以调度的节点集合。注意, 强化学习智能体在每一步都知道 $\mathcal{A}_t$, 因为输入 DAG 准确指明了哪些阶段可运行。这里, $\mathcal{A}_t$ 限制 softmax 操作所考虑的输出。整个操作端到端可微。

并行度上限选择。许多现有调度器为每个作业设置固定并行度: 例如, Spark 默认在提交作业时, 将执行器数作为命令行参数传入。Decima 每次为某个作业作出调度决策时都会调整该作业的并行度, 使并行度随着作业中不同阶段可运行或执行完成而变化。

对每个作业 i , Decima 的策略网络还使用另一个评分函数 $w(\cdot)$, 为向作业 i 分配并行度上限 l 计算得分 $w_l^i \triangleq w(y^i, z, l)$ 。与阶段选择类似, Decima 对这些得分使用 softmax 操作, 计算选择某个并行度上限的概率 (图 6)。

Decima 对所有作业和所有并行度上限复用相同的评分函数 $w(\cdot)$, 因为该函数把并行度 l 作为一个输入。如果不输入 l , 就无法区分不同并行度上限, 只能为每个上限使用独立函数。候选上限的数量可能与执行器数量一样多, 因而复用相同评分函数能显著减少策略网络参数, 并加快训练 (图 15a)。

Decima 的动作指定作业级并行度, 例如整个作业总共使用十个执行器, 而非细粒度的阶段级并行度。这一设计选择在控制粒度与模型易于训练之间作出权衡。

⁶译者注: 本例中的对数与指数变换按原文保留。通常用对数和指数构造趋近最大值的聚合时, 内层使用指数、外层使用缩放对数; 原文给出的内外变换顺序与此相反, 需结合式中的聚合顺序核查。

具体而言, 将Decima限制在作业级并行度控制, 减少了调度中必须探索和优化的调度策略空间。

不过, Decima仍然能够间接控制阶段级并行度。在每次调度事件中, Decima选择一个阶段 v , 以及该阶段所属作业的新并行度上限 l_i ; 该阶段 v 属于作业 i 。随后, 系统向 v 分配执行器, 直到 i 的并行度达到上限 l_i 。通过重复执行具有不同并行度上限的动作, Decima可以为特定阶段增加所需数量的执行器。例如, 假设作业 i 当前拥有十个执行器, 其中四个在执行阶段 v 。为给 v 再增加两个执行器, Decima在一次调度事件中选择阶段 v , 并设并行度上限为12。实验表明, Decima使用作业级并行度能够获得与细粒度阶段级并行度选择相同的性能, 同时显著加快训练 (图15a)。

5.3 训练

训练Decima的主要挑战, 是如何在持续随机作业到达的条件下训练。为解释这一挑战, 我们先介绍训练所用的强化学习算法。

强化学习训练按回合进行。每个回合包含多个调度事件, 每个事件又包含一个或多个动作。设 T 为某回合中的动作总数 (不同回合的 T 可以不同), t_k 为第 k^{th} 个动作发生的实际时刻。为指导强化学习算法, Decima在每个动作后根据高层调度目标向智能体提供奖励 r_k 。例如, 若目标是最小化平均JCT, Decima会在第 k^{th} 个动作后给予惩罚 $r_k = -(t_k - t_{k-1})J_k$, 其中 J_k 是区间 $[t_{k-1}, t_k)$ 内系统中的作业数。强化学习算法的目标是最小化惩罚的期望时间平均值: $\mathbb{E}[1/t_T \sum_{k=1}^T (t_k - t_{k-1})J_k]$ 。该目标最小化系统中的平均作业数, 因此根据Little定律[21, §5], 它实际上也最小化平均JCT。

Decima使用策略梯度算法训练。策略梯度方法的主要思想, 是利用训练期间观察到的奖励, 对神经网络参数执行梯度下降来学习。

注意, Decima从图神经网络 (第5.1节) 到策略网络 (第5.2节) 的所有操作都是可微的。为简洁起见, 将这些操作的全部参数统记为 θ , 将调度策略记为 $\pi_\theta(s_t, a_t)$, 其定义是在状态 s_t 下采取动作 a_t 的概率。

考虑长度为 T 的回合, 智能体在每一步 k 收集 (状态、动作、奖励) 观察值, 即 (s_k, a_k, r_k) 。

智能体通过REINFORCE策略梯度算法[79], 更新策略 $\pi_\theta(s_t, a_t)$ 的参数 θ :

$$\theta \leftarrow \theta + \alpha \sum_{k=1}^T \nabla_{\theta} \log \pi_{\theta}(s_k, a_k) \left(\sum_{k'=k}^T r_{k'} - b_k \right). \quad (3)$$

其中, α 为学习率, b_k 是用于降低策略梯度方差的基线[78]。例如, “基于时间”的基线[37, 53]会将 b_k 设为从第 k 步开始的累计奖励, 并对所有训练回合求平均。直观地, $(\sum_{k'} r_{k'} - b_k)$ 估计某个回合从第 k 步开始的总奖励, 比平均情况好或差多少; $\nabla_{\theta} \log \pi_{\theta}(s_k, a_k)$ 则提供参数空间中的一个方向, 使状态 s_k 下选择动作 a_k 的概率增大。

因此, 整个公式的净作用, 是提高那些能带来高于平均奖励的动作的选择概率。⁷

挑战#1: 在持续作业到达条件下训练。为了学习稳健的调度策略, 智能体在训练时必须经历作业随时间持续到达的“流式”场景。只在所有作业于回合开始时一起到达的“批量”场景中训练, 会使流式场景下的策略表现较差, 例如如图14。然而, 在连续作业到达流上训练并不容易。尤其是, 智能体的初始策略很差, 例如初始参数是随机的。

⁷式(3)的更新规则旨在最大化回合内的奖励之和。为最大化奖励的时间平均值, Decima使用该公式的一个略加修改的形式, 详见附录B。

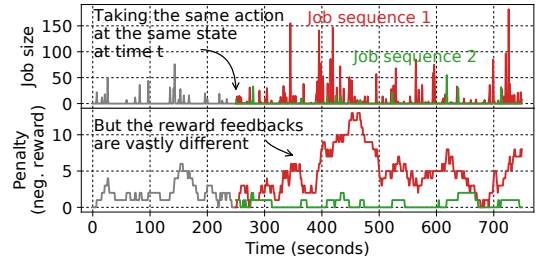


图7: 示例说明不同作业到达序列如何造成显著不同的奖励。时刻 t 之后, 我们从平均到达间隔为10秒的Poisson到达过程中, 采样两条作业到达序列; 各作业为随机采样的TPC-H查询。

图中术语: Job size 为作业规模; Job sequence 为作业序列; Time (seconds) 为时间 (秒); Penalty (neg. reward) 为惩罚 (负奖励); 图中文字说明“在时刻 t 、相同状态下采取相同动作, 但奖励反馈差异很大”。

因此, 在早期训练回合中, 智能体调度作业的速度跟不上到达速度, 几乎每个回合都会形成很长的作业队列。在这些早期回合中让智能体探索超过几步, 会浪费训练时间, 因为采用合理策略时不会出现它所遭遇的集群过载场景。

为避免这种浪费, 我们提前结束初始回合, 让智能体重置, 并尽快从空闲状态再次尝试。在整个训练过程中逐步增加回合长度。这样, 智能体起初学习调度短作业序列; 当调度策略改进后, 我们延长回合, 使问题更加困难。这种在训练中逐步增加作业序列长度、从而增加问题复杂度的思想, 实现了面向集群调度的课程学习[14]。

这一方法有个细节: 结束时刻不能是确定的。否则, 智能体可能学会预测回合何时结束, 并将某些大作业的调度推迟到结束时刻。这在固定时间范围内反而是最优策略: 回合结束时剩余作业不会受到惩罚, 因此即使令某些大作业饥饿, 严格优先调度短作业仍然更好。我们发现, 这种行为会在真实运行中造成某些作业无限期饥饿, 因为此时作业会无限持续到达。为避免这一行为, 我们使用无记忆的结束过程。具体而言, 每个训练回合在一个从指数分布随机抽取的时长 τ 后结束。如前所述, 平均回合长度在训练中逐步增大至较大值, 例如平均包含数百次作业到达。

挑战#2: 随机作业到达造成的方差。其次, 为使策略在流式场景下良好泛化, 训练回合必须包含许多不同的作业到达模式。这带来新挑战: 不同到达模式对性能的影响很大, 从而产生显著不同的奖励。例如, 考虑图7中时刻 t 的调度动作。若之后的到达序列包含一批突发的大作业, 例如序列1, 作业队列就会变长, 智能体受到的惩罚很大; 相反, 轻量作业流, 例如序列2, 会产生较短队列和较小惩罚。问题在于, 这种奖励差异与时刻 t 的动作无关, 而是由到达过程的随机性造成。强化学习算法使用奖励评估动作质量, 因此这种方差会增加噪声, 阻碍有效训练。

为解决这一问题, 我们采用近期针对“输入驱动”环境提出的方差降低技术[55]: 在这种环境中, 外生随机输入过程, 例如Decima的作业到达过程, 会影响系统动态。

其核心思想是在多个训练回合中固定使用相同的作业到达序列, 并针对每条序列分别计算基线。具体而言, 计算式(3)中的基线 b_k 时, 不再对具有不同到达序列的回合求平均, 而只对具有相同到达序列的回合求平均。在训练中, 对大量随机采样的到达序列重复这一过程。第7.2节和第7.3节介绍具体训练数据集的生成方式。该方法完全消除了到达过程所造成的方差, 使策略梯度算法能够更加准确地评估不同动作的质量, 见图14。

训练实现细节和所用超参数设置见附录C。

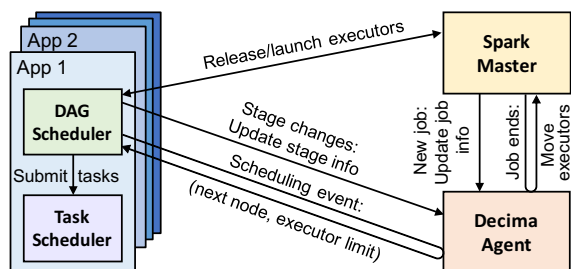


图8: Spark独立集群架构, 突出显示为Decima增加的部分。图中术语: App为应用; DAG Scheduler为DAG调度器; Task Scheduler为任务调度器; Spark Master为Spark主节点; Decima Agent为Decima智能体; Release/launch executors为释放/启动执行器; Submit tasks为提交任务; Stage changes: Update stage info为阶段变化时更新阶段信息; Scheduling event: next node, executor limit为调度事件返回下一节点及执行器上限; New job: Update job info为新作业到达时更新作业信息; Job ends: Move executors为作业结束时迁移执行器。

6 实现

我们将Decima实现为可插拔的调度服务, 并行数据处理平台可以通过RPC接口与之通信。

第6.1节介绍Decima与Spark的集成。

随后介绍基于Python的训练基础设施, 包括一个精确的Spark集群仿真器(第6.2节)。

6.1 与Spark集成

Spark集群⁸运行多个并行应用, 每个应用包含一个或多个作业, 共同组成由处理阶段构成的DAG。

Spark主节点管理应用执行, 并监控多个工作节点的健康状态; 每个工作节点将资源划分给多个执行器。

执行器为某个特定应用创建, 并始终与该应用关联; 该应用自行将工作调度给执行器。

应用完成后, 其执行器终止。图8展示这一架构。

为将Decima集成到Spark, 我们作了两项主要修改:

- (1) 各应用的DAG调度器在启动和每次调度事件发生时联系Decima。
Decima返回接下来处理的阶段和并行度上限(第5.2节)。
- (2) Spark主节点在新作业到达时联系Decima, 确定为它启动多少执行器, 并在执行器完成某个阶段后从作业中收回执行器, 协助Decima进行资源分配。

状态观察。在Decima中, 作业DAG i 中节点 o 的特征向量 x_o^i (第5.1节) 包含: (i) 该阶段剩余任务数; (ii) 平均任务时长; (iii) 当前在该节点上工作的执行器数; (iv) 可用执行器数; (v) 可用执行器是否位于该作业本地。

我们选择这些特征, 是为了尽可能涵盖刻画集群状态所必需的信息, 例如当前分配给各阶段的执行器数, 以及有助于调度决策的统计信息, 例如阶段的平均任务时长。

这些统计信息取决于可用信息, 例如同一个作业此前执行的性能画像或运行时指标, 也取决于所用系统; 这里是Spark。

Decima可以方便地加入其他信号。

神经网络架构。图神经网络的六个变换函数 $f(\cdot)$ 和 $g(\cdot)$ (第5.1节, 节点、作业和全局嵌入各两个), 以及策略网络的两个评分函数 $q(\cdot)$ 和 $w(\cdot)$ (第5.2节), 均用双隐藏层神经网络实现, 两层分别包

⁸这里讨论Spark的“独立”运行模式 (<http://spark.apache.org/docs/latest/spark-standalone.html>)。基于YARN的部署原则上也能使用Decima, 但需要同时修改Spark和YARN。

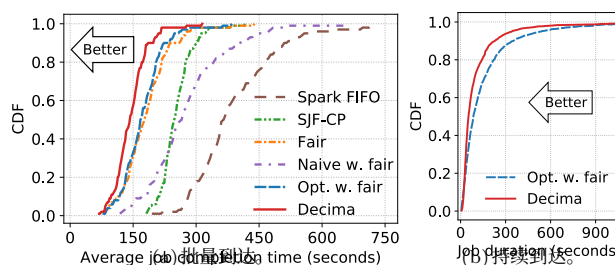


图9: 在真实Spark集群上, 针对批量或持续到达的TPC-H作业, Decima学习得到的调度策略相比基线算法, 将平均作业完成时间缩短21%至获得3.1×的改善。

图中术语: CDF为累积分布函数; Average job completion time, Job duration为平均作业完成时间、作业时长, 单位秒; Better的箭头指向较好结果; Fair、Naive w. fair、Opt. w. fair分别为公平、朴素加权公平、最优加权公平; 其余算法名称保持原样。

含32和16个隐藏单元。这些网络在全部作业和全部并行度上限之间复用, 因此Decima模型很轻量, 总共只有12,736个参数, 约50KB。将集群状态映射为调度决策的耗时小于15毫秒(图15b)。

6.2 Spark仿真器

Decima在离线环境中训练, 使用高保真仿真器, 读取真实Spark集群的性能画像, 例如任务时长(第7.2节), 以及工业轨迹中的作业运行时间特征(第7.3节)。

为忠实模拟Decima决策与集群之间的相互作用, 仿真器刻画以下真实系统效应:

- (1) 某个阶段的第一“波”任务, 经常比后续任务运行得慢。
这来自Spark的流水线任务执行[63]、任务代码的JIT编译[47], 以及建立到其他执行器的TCP连接等预热成本。因此, Decima的仿真环境从与后续波次不同的分布中抽取首波任务的实际运行时间。
- (2) 向Spark作业增加执行器需要启动JVM进程, 耗时2–3秒。执行器与作业绑定, 以实现隔离, 而且Spark假定执行器会长期存活。
因此, 每次Decima在作业之间迁移执行器时, 仿真环境都会加入反映启动延迟的空闲时间。
- (3) 较高并行度可能减慢单个Spark任务, 因为更大范围的shuffle需要更多TCP连接, 合并许多分片的数据时也会增加工作量。
若有相应数据, Decima的环境便从不同并行度下采集的任务时长分布中采样, 以刻画这些影响。

附录D通过与真实Spark执行比较, 验证仿真器的保真度。

7 评估

我们在真实Spark集群测试平台, 以及使用阿里巴巴生产工作负载的仿真中评估Decima。

实验回答以下问题:

- (1) 在真实Spark集群中, Decima相比精心调优的启发式方法表现如何(第7.2节)?
- (2) Decima的学习能否泛化到机器配置不同的多资源场景(第7.3节)?
- (3) 各个关键设计分别如何影响Decima的性能? 调度环境变化时Decima如何适应? 训练速度与训练后的调度决策速度如何?

7.1 已有基线算法

评估中, 我们将Decima与以下七种基线算法比较:

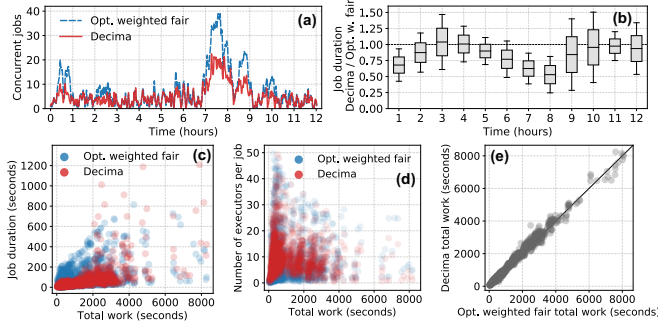


图 10: 对持续到达 Spark 集群的 TPC-H 作业进行时间序列分析 (a, b), 表明 Decima 相对启发式方法的大部分性能收益出现在繁忙时段, 例如在第 8 小时使作业运行快 2x。它为小作业赋予适当优先级 (c), 分配更多执行器 (d), 同时防止工作量膨胀 (e)。

图中术语: Concurrent jobs 为并发作业数; Time (hours) 为时间 (小时); Job duration 为作业时长; Total work 为总工作量, 单位秒; Job duration: Decima / Opt. w. fair 为两种策略的作业时长之比; Number of executors per job 为每作业执行器数; Decima total work, Opt. weighted fair total work 为两种策略的总工作量; Opt. weighted fair 为最优加权公平。

- (1) Spark 默认 FIFO 调度, 按到达顺序运行作业, 并为每个作业分配用户请求数量的执行器。
- (2) 最短作业优先关键路径启发式方法 (SJF-CP), 根据总工作量确定作业优先级, 并在每个作业内运行关键路径上下一个阶段的任务。
- (3) 简单公平调度, 给每个作业同等的执行器份额, 并在可运行阶段的任务间轮转, 让所有分支同时推进。
- (4) 朴素加权公平调度, 按作业总工作量的比例分配执行器。
- (5) 精心调优的加权公平调度, 为每个作业分配全部执行器的 $T_i^\alpha / \sum_i T_i^\alpha$; 其中 T_i 为作业 i 的总工作量, α 是调节因子。注意, $\alpha=0$ 时退化为简单公平调度, $\alpha=1$ 时为朴素加权公平调度。我们遍历 $\alpha \in \{-2, -1.9, \dots, 2\}$, 寻找最优因子。
- (6) Tetris 的标准多资源装箱算法 [34], 贪心地选择使请求资源向量与可用资源向量点积最大的阶段。
- (7) Graphene*, 即面向 Decima 离散执行器类别改造的 Graphene [36]。Graphene* 使用 Graphene 算法 [36, §4.1] 识别并分组“棘手”节点, 再像第 (5) 项一样, 用最优调节的并行度一起调度这些节点, 保留 Graphene 规划策略的核心。我们用网格搜索优化超参数, 详见附录 F。

7.2 Spark 集群

我们使用 ChameleonCloud 测试平台中的 OpenStack 集群, 运行按第 6.1 节修改的 Spark v2.2。⁹

集群包含 25 个工作节点 VM, 每个 m1.xlarge 实例具有 8 个 CPU、16GB 内存, 运行两个执行器; 主节点 VM 为 m1.xxxlarge 实例, 具有 16 个 CPU 和 32GB 内存。

实验考虑两种到达方式: (i) 批量到达, 多个作业同时开始并运行至结束; (ii) 持续到达, 作业按随机到达间隔分布或轨迹到达。

批量到达。我们从六种输入规模 (2、5、10、20、50、100GB) 及全部 22 种 TPC-H 查询 [73] 中随机采样作业, 产生重尾分布: 23% 的作业包含总工作量的 82%。

⁹<https://www.chameleoncloud.org>

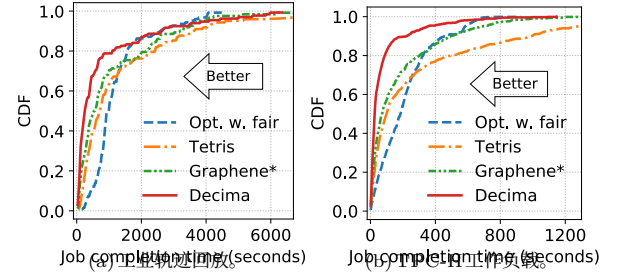


图 11: 在多维资源条件下, Decima 的调度策略相比 Graphene* 将平均 JCT 缩短 32% 至 43%。

图中术语: CDF 为累积分布函数; Job completion time (seconds) 为作业完成时间 (秒); Better 的箭头指向较好结果; Opt. w. fair 为最优加权公平; 其余算法名称保持原样。

每批包含 20 个训练中未见过的随机作业, 同时到达; 我们测量其平均 JCT。

图 9a 展示 100 次实验的平均 JCT 累积分布。

结果包含三个关键观察。首先, SJF-CP 和公平调度虽然简单, 平均表现却分别比 FIFO 好 1.6x 和 2.5x。

公平调度策略优于 SJF-CP, 是因为它们同时推进多个作业, 而 SJF-CP 将全部执行器集中在最短作业上。

其次, 一个可能出乎意料的结果是: 不加权公平调度, 优于按作业规模加权的公平调度, 即“朴素加权公平”策略。

这是因为加权公平调度分给小作业的执行器少于同等份额, 使小作业变慢并增大平均 JCT。

我们调优后的加权公平启发式方法, 即“最优加权公平”, 通过在每次实验中校准各作业权重, 抵消这一影响 (第 7.1 节)。

最优 α 通常约为 -1, 即将执行器数设置为与作业规模成反比。该策略起初有效地聚焦小作业, 之后转而并行运行大作业; 相比公平调度, 性能提升 11%。

最后, Decima 优于全部基线算法, 相比最接近的启发式方法“最优加权公平”, 平均 JCT 缩短 21%。

原因是 Decima 更好地设定作业优先级, 为不同作业分配高效的执行器份额, 并利用作业 DAG 结构。第 7.4 节分解这些因素各自的收益。

Decima 通过端到端强化学习训练自主学习该策略, 而表现最好的基线算法需要精心调优。

持续到达。我们均匀随机采样六种规模的 1,000 个 TPC-H 作业, 并将到达建模为平均间隔 45 秒的 Poisson 过程。

相应集群负载约为 85%。在这一负载下, 作业到达速度高于大多数启发式调度策略完成它们的速度。

图 9b 表明, Decima 优于唯一能够跟上到达速度的基线算法“最优加权公平”; 其平均 JCT 低 29%。

Decima 在繁忙的高负载时段尤其有效, 此时调度决策的影响远大于集群资源充足时。

图 10a 表明, Decima 的系统中的并发作业数量始终低于调优后的启发式方法, 尤其是在第 7-9 小时的繁忙时段; 此时 Decima 完成作业的速度约快 2x (图 10b)。

高负载性能对批处理集群很重要, 因为它们经常具有很长的作业队列 [66]; 高负载时段也是良好调度决策影响最大的时段, 例如可减少为应对工作负载峰值而进行的过量资源配置。

Decima 的收益来自更快完成小作业, 图 10c 左下角密集的红点展示了这一点。

Decima 通过给小作业分配更多执行器实现这一效果 (图 10d)。

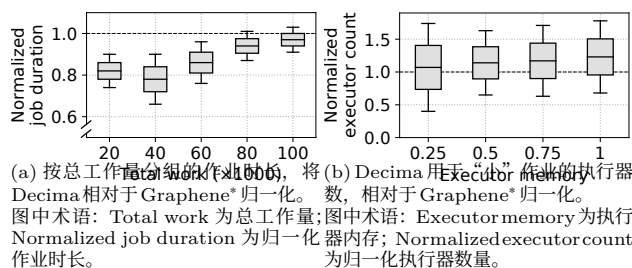


图 12: 在多维资源条件下, Decima 通过 (a) 更快完成小作业, 以及 (b) 为小作业使用“过大”的执行器, 优于 Graphene*。这里小作业指总工作量最小的 20%。

各作业合适的执行器数取决于工作负载: 不加区分地给小作业更多执行器, 会低效使用集群资源 (第 2.2 节)。

例如, SJF-CP 严格地将所有可用执行器分配给最小作业, 但这种低效分配使总工作量膨胀, 因此 SJF-CP 会形成不断增长的作业积压。

相对而言, Decima 的执行器分配产生的总工作量, 与人工调优的启发式方法相近。

图 10e 展示了这一点: 对角线以下的作业在 Decima 中的总工作量低于启发式方法, 以上的作业则在 Decima 中工作量更大。

多数小作业位于对角线上, 说明 Decima 只有在额外执行器仍能高效工作时才提高并行度上限。

因此, Decima 成功平衡了两件事: 给小作业更多资源以尽早完成, 以及高效利用资源。

7.3 多维资源装箱

前述实验使用的 Spark 独立调度器, 只允许作业访问预先定义的执行器槽位。

YARN [75]、Mesos [41] 等更高级集群调度器, 允许作业指定任务资源需求, 并创建大小合适的执行器。

将具有多维资源需求 (例如, (CPU、内存)) 的任务装入容量固定的服务器, 会进一步增加调度问题的复杂性 [34, 36]。

我们用阿里巴巴生产轨迹, 研究 Decima 能否采用相同核心方法学习良好的多维调度策略。

工业轨迹。轨迹包含某个生产集群中的约 20,000 个作业。许多作业具有复杂 DAG: 59% 的作业包含四个或更多阶段, 有些则包含数百个阶段。我们用仿真器 (第 6.2 节) 运行实验, 执行器数量最多为 30,000。

这一参数根据轨迹中的最大并发任务数设置。我们用轨迹前半部分训练, 再在剩余部分将 Decima 与其他方案比较。

多资源环境。我们修改 Decima 环境, 提供内存大小不同的多种离散执行器类别。

此时任务要求最低数量的 CPU 和内存, 即运行任务的执行器必须能容纳它。

任务可以运行在资源量大于或等于其资源请求的执行器上。Decima 此时需要选择待调度的 DAG 阶段、并行度和执行器类别。

实验使用四种执行器, 每种具有 1 个 CPU 核心, 归一化内存分别为 (0.25, 0.5, 0.75, 1) 单位; 每种类别占集群执行器总数的 25%。

结果。我们在仿真中进行多资源实验, 作业按轨迹持续到达。

图 11a 展示 Decima 与另外三种算法的结果: 最优调优的加权公平调度方法、Tetris 和 Graphene*。

Decima 的平均 JCT 比表现最好的竞争算法 Graphene* 低 32%, 说明它在多资源环境中学到了良好策略。

Decima 的策略与 Graphene* 在性质上不同。图 12a 按作业总工作量分解 Decima 相比 Graphene* 的改进。

Decima 对所有规模的作业都比 Graphene* 完成得更快, 但小作业的收益尤其大。

原因是: 当大小刚好合适的执行器数量不足时, Decima 学会用“过大”的执行器, 帮助那些即将完成的小作业尽快结束。

图 12b 说明这一点: 对总工作量最小的 20% 作业, Decima 使用的最大类别执行器多 39%; 完整分析见附录 G。

换言之, Decima 在内存碎片与更快清空作业队列之间作出权衡。

这种权衡合理, 因为小作业 (i) 对平均 JCT 目标的贡献更大, 且 (ii) 只会短时间造成资源碎片。

相对而言, Tetris 贪心地将任务放入最合适的执行器类别, 实现最低内存碎片。

Decima 的资源碎片相对 Tetris 的差异在 4%–13% 之内, 但其平均 JCT 低 52%, 因为它学会了较好地平衡上述权衡。

这要求考虑依赖工作负载的因素, 例如 DAG 结构、“小”作业的判定阈值等。

Graphene* 等启发式方法试图用加权评分函数和大量调优平衡这些因素; Decima 无需这些人工输入就能学习。

我们还用 TPC-H 工作负载重复该实验, 使用 200 个执行器, 并从 (0,1] 中采样每个 TPC-H DAG 节点的内存请求。

图 11b 表明, Decima 对其他算法的优势更大, 例如相比 Graphene* 改善 43%。

这是因为工业轨迹缺少不同并行度下工作量膨胀的测量, 而我们为 TPC-H 提供了这些测量。

Decima 学会利用这些信息进一步校准执行器分配。

7.4 深入分析 Decima

最后, 我们展示 Decima 能学习的多种调度策略, 并分解关键思想与技术对其性能的影响。

在附录中, 我们还通过穷举作业顺序评估 Decima 的最优性 (附录 H), 考察学习策略对环境变化的稳健性 (附录 I), 以及对不完整信息的敏感性 (附录 J)。

学到的策略。Decima 优于其他算法, 是因为它能够根据高层目标、工作负载和环境条件学习不同策略。

当优化平均 JCT 时 (图 13a), Decima 学会为小作业共享执行器, 使它们尽快结束, 并避免给大作业使用过多执行器而降低效率 (第 7.2 节)。

Decima 也会让执行器持续处理同一作业的任务, 以避免迁移开销 (第 6.1 节)。

但如果执行器在作业间迁移没有开销, 例如任务很长时迁移开销可以忽略, 或系统没有 JVM 启动开销, Decima 就会学到积极在作业间迁移执行器的策略。图 13b 中频繁的颜色变化展示了这一点。

最后, 如果目标改为最小化一批作业整体的总完工时间, Decima 又会学到另一种策略 (图 13c)。

总完工时间目标只关心最后一个作业的完成时间, 因此 Decima 不再努力让各作业尽早完成。

相反, 许多作业在批量工作负载的末尾一同完成, 使调度器在整个执行过程中始终拥有更多作业选择, 提高集群利用率。

学习架构的影响。我们选择性移除组件, 以验证 Decima 是否利用了状态中的全部原始信息, 以及是否需要各个关键设计组件。

在不同负载的仿真集群上, 我们运行持续到达的 1,000 个 TPC-H 作业, 并为每种负载训练五种不同的 Decima 变体。

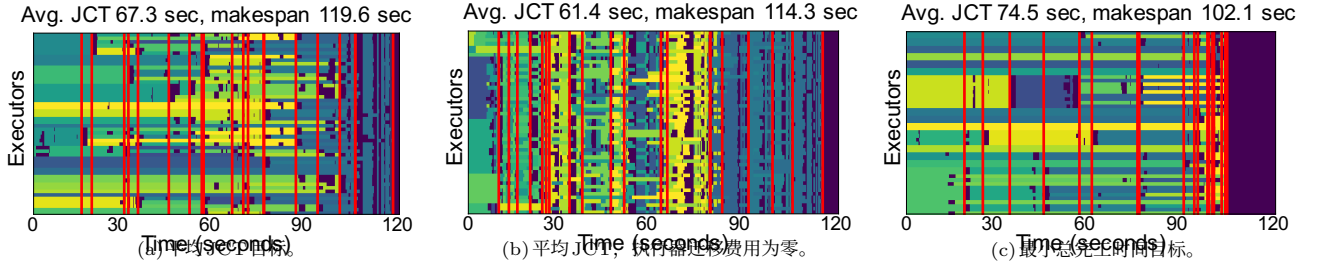


图 13: Decima 根据环境和目标学习性质不同的策略: 例如执行器迁移有开销 (a) 或无开销 (b); 优化平均 JCT (a) 或总完工时间 (c)。红色竖线表示作业完成, 不同颜色表示不同作业的任务, 深紫色为空闲时间。

图中术语: Executors 为执行器; Time (seconds) 为时间 (秒); Avg. JCT 为平均作业完成时间; makespan 为总完工时间; sec 为秒。

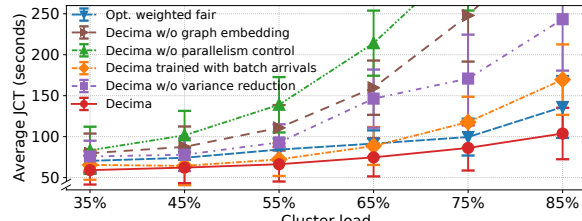


图 14: 持续作业到达时, 各关键思想对 Decima 的贡献分解。移除任何一种思想, 都会使 Decima 的平均 JCT 高于加权公平策略。

图中术语: Cluster load 为集群负载; Average JCT (seconds) 为平均作业完成时间 (秒); Opt. weighted fair 为最优加权公平; w/o graph embedding 为不使用图嵌入; w/o parallelism control 为不控制并行度; trained with batch arrivals 为用批量到达训练; w/o variance reduction 为不降低方差。

图 14 表明, 在高集群负载下, 移除任何组件都会使 Decima 的平均 JCT 比调优后的加权公平启发式方法更差。

这一结果有四点启示。首先, 并行度控制对 Decima 性能影响最大。没有并行度控制时, Decima 在每次调度事件中把所有可用执行器分配给一个阶段。

即使集群只有中等负载, 例如 55%, 这也会产生不稳定策略, 跟不上新作业的到达率。

其次, 移除图嵌入, 即将各节点的原始特征作为第 5.2 节评分函数的输入, 会使 Decima 无法估计作业剩余工作量, 也无法考虑集群中的其他作业。

因此, Decima 不知道什么是小作业, 也不知道集群负载, 其学到的策略随负载增大很快变得不稳定。

第三, 在各训练回合间使用不固定的作业序列, 会增大奖励信号的方差 (第 5.3 节)。

负载越高, 作业到达序列越多样, 奖励方差也随之增大。

集群负载超过 75% 时, 在训练 Decima 中通过同步终止来降低这一方差, 使平均 JCT 改善 2x。这说明, 在长时间范围的调度问题中, 降低方差是学习高质量策略的关键。

第四, 只用批量作业到达训练, 不能泛化到持续作业到达。

用批量到达训练时, Decima 学会系统性地推迟大作业, 因为这样会使 JCT 总和、即惩罚总和最低。

持续到达时, 负载越大、作业到达越频繁, 这一策略就会让大作业无限期饥饿。

因此, 只用批量到达训练时, Decima 在超过 65% 的负载下, 表现劣于调优的加权公平启发式方法。

泛化到不同工作负载。 我们修改 TPC-H 实验中的训练工作负载, 测试 Decima 的泛化能力 (第 7.2 节)。为模拟集群工作负

配置 (IAT: 到达间隔)	平均 JCT (秒)
最优加权公平 (最佳启发式)	91.2±23.5
Decima, 按测试负载训练 (IAT: 45 秒)	65.4±28.7
Decima, 按“反偏斜”负载训练 (IAT: 75 秒)	104.8±37.6
Decima, 混合负载训练	82.3±31.2
Decima, 混合负载训练并提供到达间隔提示	76.6±33.4

表 2: Decima 泛化到变化的工作负载。对于未见过的的工作负载, 混合负载训练使 Decima 比最佳启发式方法改善 10%; 如果通过输入特征知道到达间隔, 则改善 16%。

载变化, 我们用作业到达间隔为 42 至 75 秒的不同负载训练模型, 再用到达间隔为 45 秒的负载测试。

Decima 学习的是针对特定工作负载的策略, 因此我们预期其有效性取决于测试负载的整体特征, 例如到达间隔和作业规模分布, 是否与训练负载相符。

表 2 给出相应平均 JCT。用与测试负载相似的负载训练时, Decima 表现较好。而用到达间隔为 75 秒的“反偏斜”负载训练时, Decima 泛化较差, 表现劣于优化后的加权公平策略, 这并不意外。

这是合理的, 因为 Decima 在策略中利用了学到的到达间隔分布。

在覆盖整个到达间隔范围的混合负载集上训练时, Decima 能学到更通用的策略。

该策略不再强烈拟合某一种到达间隔分布, 因此对工作负载变化更加稳健。

如果 Decima 能观察到到达间隔, 并将其作为状态特征 (第 6.1 节), 泛化还会更好; 学到的自适应策略使平均 JCT 比最佳启发式方法低 16%。

这些结果表明, 多样的训练负载有助于使 Decima 的策略抵御负载变化; 第 8 节讨论可能的在线学习。

训练与推理性能。 图 15a 中的蓝线展示 Decima 在持续到达的 TPC-H 作业上的学习曲线 (第 7.2 节)。每 100 次迭代, 使用未见过的作业到达序列测试一次模型快照。

每次训练迭代约耗时 5 秒。Decima 的设计 (第 5.3 节) 对训练效率至关重要: 省略输入中的并行度上限值 (黄线), 迫使 Decima 为不同上限使用独立评分函数, 显著增加需优化的参数; 采用节点级细粒度并行度控制 (绿线), 则增大必须探索的算法空间, 使训练变慢。

图 15b 展示 Spark 测试平台中 (第 7.2 节), Decima 决定一个调度动作所需时间的累积分布 (红线) 以及调度事件间隔的累积分布 (蓝线)。

Decima 的平均调度延迟小于 15 毫秒, 而调度事件间隔通常是秒级。

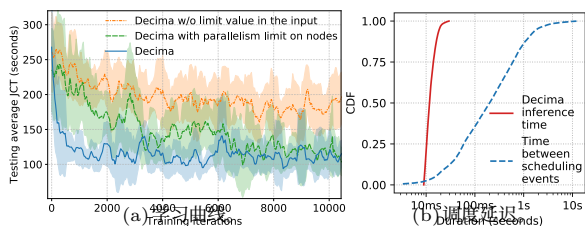


图 15: 作业并行度的不同编码方式 (第 5.2 节) 影响 Decima 的训练时间。Decima 能低延迟地作出调度决策: 平均而言, 其延迟约为调度事件间隔的五十分之一, 即相差约 50 $\times$ 。

图中术语: Training iterations 为训练迭代数; Testing average JCT (seconds) 为测试平均 JCT (秒); w/o limit value in the input 为输入中不含并行度上限值; with parallelism limit on nodes 为在节点上设置并行度上限; CDF 为累积分布函数; Duration (seconds) 为时长 (秒); Decima inference time 为 Decima 推理时间; Time between scheduling events 为调度事件间隔。

在少于 5% 的情况下, 调度间隔短于调度延迟, 例如集群在单个调度事件中请求多个调度动作时。因此, Decima 的调度延迟不会给任务运行时间带来可测量的开销。

8 讨论

本节讨论未来研究方向, 以及 Decima 技术的其他潜在应用。

稳健性与泛化。第 7.4 节的实验表明, Decima 能学习可泛化的调度策略, 在未见过的工作负载上表现良好。

然而, 实际变化可能比到达间隔变化更剧烈。

为增强策略对这类变化的稳健性, 可以借鉴新兴的稳健对抗强化学习研究[64], 在最坏情形或对抗性工作负载上训练智能体。

另一方向是随工作负载变化, 在线调整调度策略。

在线方法的关键挑战, 是当工作负载快速变化时, 降低无模型强化学习较大的样本复杂度。

一种可行方式可能是元学习[22, 27, 29], 训练一个“元”调度智能体, 使其只需少量观察就能适应特定工作负载。

其他学习目标。实验使用了与作业时长相关的指标评估 Decima, 例如平均 JCT 和总完工时间。

以不同方式塑造奖励信号, 也可以引导 Decima 满足其他目标。

例如, 每当作业错过截止期时施加严重惩罚, 可以引导 Decima 学习感知截止期的策略。

或者, 以作业时长经验样本的第 90 百分位数等指标为奖励依据, Decima 可以优化 JCT 分布的尾部, 使其更紧。使用强化学习解决约束优化目标, 例如最小化平均 JCT 同时严格保证公平性, 是有趣的进一步方向[2, 30]。

抢占式调度。Decima 当前从不抢占运行中的任务, 只有阶段完成后才能从作业中移除执行器。

这一设计使 MDP 对强化学习而言保持可处理, 产生有效学习及强大的调度策略。

不过, 未来可以研究在 Decima 这类强化学习驱动调度器中, 实现更细粒度、反应更及时的抢占。

直接引入抢占, 会产生大得多的动作空间, 例如指定任意需抢占的执行器集合, 并且可能要求远高于当前的决策频率。

为使强化学习问题保持可处理, 一种潜在研究方向是使用多智能体强化学习[38, 50, 62]。

例如, 一个类似 Decima 的调度智能体控制接下来运行哪个阶段、分配多少执行器; 与此同时, 另一个智能体决定抢占哪些执行器。

潜在网络与系统应用。我们为 Decima 开发的某些技术, 也广泛适用于其他网络及计算机系统问题。例如, 输入 DAG 的可扩展表示 (第 5.1 节) 可用于图上的问题, 如数据库查询优化[56]和硬件设备放置[3]。方差降低技术 (第 5.3 节) 则一般适用于具有随机、不可预测输入的系统[54, 55]。

9 相关工作

此前很少有工作将机器学习技术应用于集群调度。

DeepRM [53] 用强化学习训练神经网络, 完成多维资源装箱; 它在目标和方法上最接近 Decima。

但 DeepRM 只处理每个作业为单个任务的基本场景, 并在简单仿真环境中评估。

其学习模型也不支持具有 DAG 结构的作业, 训练过程无法处理作业持续到达的真实集群负载。

在其他应用中, Mirhoseini 等人学习 TensorFlow (TF) 计算的设备放置[60], 也使用强化学习, 但依赖循环神经网络遍历全部节点来生成状态嵌入, 而非图神经网络。

该方法使用循环神经网络遍历全部节点以嵌入状态, 而不是使用可扩展图神经网络。

其目标是调度好单个 TF 作业, 模型无法泛化到未见过的作业组合[59]。

此前机器学习和算法设计工作, 将强化学习与图神经网络结合, 优化顶点集合覆盖、旅行商等复杂组合问题[23, 49]。

Decima 的可扩展状态表示受这条研究路线启发, 但我们发现现成图神经网络在本问题上表现较差。

为训练强大的调度智能体, 我们必须修改图神经网络架构, 使 Decima 能计算 DAG 关键路径等指标 (第 5.1 节)。

在更广泛的资源管理系统中, Paragon[25]和 Quasar[26]使用协同过滤, 将工作负载匹配到不同机器类型并避免干扰; 他们的目标与 Decima 互补。

Tetrisched[74]与 Decima 一样提前规划, 但用约束求解器优化作业放置, 并要求用户随作业提供显式约束。

Firmament[33]也使用约束求解器, 实现高质量放置, 但要求管理员配置复杂的调度策略。

Graphene[36]用启发式方法调度作业 DAG, 却不能设定合适的并行度。

一些系统通过“自动扩缩容”并行度来满足作业截止期[28], 或机会性地利用空闲资源加速作业[68, §5]。

Carbyne[35]允许作业“利他性”地放弃部分短期公平资源份额, 改善不同作业的 JCT, 同时保证长期公平性。Decima 学习到与 Carbyne 类似的策略, 平衡资源份额与装箱来降低平均 JCT; 但 Decima 当前设计并没有将公平性列为目标。

Borg[77]、Mesos[41]或 YARN[75]等通用集群管理器支持许多不同应用, 因而在这一层面应用针对具体工作负载的调度策略较为困难。

不过, Decima 可以作为运行在 Mesos 或 Omega[68]之上的框架。

10 结论

Decima 表明, 使用强化学习自动学习复杂集群调度策略是可行的; 学到的策略既灵活又高效。

Decima 的图嵌入技术、面向流式场景的训练框架等学习创新, 也可能适用于查询优化器等其他处理 DAG 的系统。

我们将开源 Decima、模型及实验基础设施, 网址为 <https://web.mit.edu/decima>。

本工作不涉及伦理问题。

致谢。感谢论文指导审稿人 Aditya Akella 和 SIGCOMM 审稿人提供宝贵反馈。感谢 Akshay Narayan、Amy Ousterhout、Prateesh Goyal、Peter Iannucci 和 Songtao He，在项目开发期间进行富有成效的讨论。感谢阿里云智能的 Haiyang Ding 和 Yihui Feng 分享生产集群数据集。本工作部分获得 NSF CNS-1751009、CNS-1617702 项目、Google 教师研究奖、AWS 机器学习研究奖、Cisco 研究中心奖、Alfred P. Sloan 研究奖学金，以及 MIT DSAIL 实验室赞助方资助。

参考文献

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-scale Machine Learning. In Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI). 265–283. <http://dl.acm.org/citation.cfm?id=3026877.3026899>
- [2] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. 2017. Constrained policy optimization. In Proceedings of the 34th International Conference on Machine Learning-Volume 70. 22–31.
- [3] Ravichandra Addanki, Shaileshh Bojja Venkatakrishnan, Shreyan Gupta, Hongzi Mao, and Mohammad Alizadeh. 2018. Placeto: Efficient Progressive Device Placement Optimization. In Proceedings of the 1st Machine Learning for Systems Workshop.
- [4] Sameer Agarwal, Srikanth Kandula, Nicolas Bruno, Ming-Chuan Wu, Ion Stoica, and Jingren Zhou. 2012. Re-optimizing Data-parallel Computing. In Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation (NSDI). 281–294. <http://dl.acm.org/citation.cfm?id=2228298.2228327>
- [5] Kunal Agrawal, Jing Li, Kefu Lu, and Benjamin Moseley. 2016. Scheduling parallel DAG jobs online to minimize average flow time. In Proceedings of the 27th annual ACM-SIAM symposium on Discrete Algorithms (SODA). Society for Industrial and Applied Mathematics, 176–189.
- [6] Alibaba. 2017. Cluster data collected from production clusters in Alibaba for cluster management research. <https://github.com/alibaba/clusterdata>. (2017).
- [7] Dario Amodei and Danny Hernandez. 2018. AI and Compute. <https://openai.com/blog/ai-and-compute/>. (2018).
- [8] Apache Hadoop. 2014. Hadoop Fair Scheduler. (2014). http://hadoop.apache.org/common/docs/stable1/fair_scheduler.html
- [9] Apache Spark. 2018. Spark: Dynamic Resource Allocation. (2018). <http://spark.apache.org/docs/2.2.1/job-scheduling.html#dynamic-resource-allocation> Spark v2.2.1 Documentation.
- [10] Apache Tez 2013. Apache Tez Project. <https://tez.apache.org/>. (2013).
- [11] Luiz André Barroso, Jimmy Clidaras, and Urs Hölzle. 2013. The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines, second edition. Synthesis Lectures on Computer Architecture 8, 3 (July 2013). <https://doi.org/10.2200/S00516ED2V01Y201306CAC024>
- [12] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Flores Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Çağlar Gülçehre, Francis Song, Andrew J. Ballard, Justin Gilmer, George E. Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matthew Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. 2018. Relational inductive biases, deep learning, and graph networks. arXiv preprint arXiv:1806.01261 (2018).
- [13] Richard Bellman. 1966. Dynamic programming. Science 153, 3731 (1966), 34–37.
- [14] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In Proceedings of the 26th annual International Conference on Machine Learning (ICML). 41–48.
- [15] Dimitri P Bertsekas and John N Tsitsiklis. 1995. Neuro-dynamic programming: an overview. In Decision and Control, 1995., Proceedings of the 34th IEEE Conference on, Vol. 1. IEEE, 560–564.
- [16] Arka A. Bhattacharya, David Culler, Eric Friedman, Ali Ghodsi, Scott Shenker, and Ion Stoica. 2013. Hierarchical Scheduling for Diverse Datacenter Workloads. In Proceedings of the 4th Annual Symposium on Cloud Computing (SoCC). Article 4, 15 pages. <https://doi.org/10.1145/2523616.2523637>
- [17] Christopher M. Bishop. 2006. Pattern Recognition and Machine Learning. Springer.
- [18] Robert D Blumofe and Charles E Leiserson. 1999. Scheduling multithreaded computations by work stealing. Journal of the ACM (JACM) 46, 5 (1999), 720–748.
- [19] Craig Chambers, Ashish Raniwala, Frances Perry, Stephen Adams, Robert R. Henry, Robert Bradshaw, and Nathan Weizenbaum. 2010. FlumeJava: Easy, Efficient Data-parallel Pipelines. In Proceedings of the 2010 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). 363–375. <https://doi.org/10.1145/1806596.1806638>
- [20] Chandra Chekuri, Ashish Goel, Sanjeev Khanna, and Amit Kumar. 2004. Multi-processor scheduling to minimize flow time with ϵ resource augmentation. In Proceedings of the 36th Annual ACM Symposium on Theory of Computing. 363–372.
- [21] Dilip Chhajed and Timothy J Lowe. 2008. Building intuition: insights from basic operations management models and principles. Vol. 115. Springer Science & Business Media.
- [22] Ignasi Clavera, Jonas Rothfuss, John Schulman, Yasuhiro Fujita, Tamim Asfour, and Pieter Abbeel. 2018. Model-based reinforcement learning via meta-policy optimization. arXiv preprint arXiv:1809.05214 (2018).
- [23] Hanjun Dai, Elias B. Khalil, Yuyu Zhang, Bistra Dilkina, and Le Song. 2017. Learning Combinatorial Optimization Algorithms over Graphs. In Proceedings of the 31st Conference on Neural Information Processing Systems (NeurIPS). 6348–6358.
- [24] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. arXiv preprint arXiv:1606.09375 (2016).
- [25] Christina Delimitrou and Christos Kozyrakis. 2013. Paragon: QoS-aware Scheduling for Heterogeneous Datacenters. In Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 77–88. <https://doi.org/10.1145/2451116.2451125>
- [26] Christina Delimitrou and Christos Kozyrakis. 2014. Quasar: Resource-efficient and QoS-aware Cluster Management. In Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 127–144. <https://doi.org/10.1145/2541940.2541941>
- [27] Yan Duan, John Schulman, Xi Chen, Peter L Bartlett, Ilya Sutskever, and Pieter Abbeel. 2016. RL2: Fast Reinforcement Learning via Slow Reinforcement Learning. arXiv preprint arXiv:1611.02779 (2016).
- [28] Andrew D Ferguson, Peter Bodik, Srikanth Kandula, Eric Boutin, and Rodrigo Fonseca. 2012. Jockey: guaranteed job latency in data parallel clusters. In Proceedings of the 7th ACM European Conference on Computer Systems (EuroSys).
- [29] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. In Proceedings of the 34th International Conference on Machine Learning (ICML). 1126–1135.
- [30] Peter Geibel. 2006. Reinforcement learning for MDPs with constraints. In Proceedings of the 17th European Conference on Machine Learning (ECML). 646–653.
- [31] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. 2011. Dominant Resource Fairness: Fair Allocation of Multiple Resource Types. In Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI). 323–336. <http://dl.acm.org/citation.cfm?id=1972457.1972490>
- [32] Ali Ghodsi, Matei Zaharia, Scott Shenker, and Ion Stoica. 2013. Choosy: max-min fair sharing for datacenter jobs with constraints. In Proceedings of the 8th ACM European Conference on Computer Systems (EuroSys). 365–378. <https://doi.org/10.1145/2465351.2465387>
- [33] Ionel Gog, Malte Schwarzkopf, Adam Gleave, Robert N. M. Watson, and Steven Hand. 2016. Firmament: fast, centralized cluster scheduling at scale. In Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 99–115.
- [34] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. 2014. Multi-resource Packing for Cluster Schedulers. In Proceedings of the 2014 ACM SIGCOMM Conference (SIGCOMM). 455–466.
- [35] Robert Grandl, Mosharaf Chowdhury, Aditya Akella, and Ganesh Ananthanarayanan. 2016. Altruistic Scheduling in Multi-resource

- Clusters. In Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI). 65–80. <http://dl.acm.org/citation.cfm?id=3026877.3026884>
- [36] Robert Grandl, Srikanth Kandula, Sriram Rao, Aditya Akella, and Janardhan Kulkarni. 2016. Graphene: Packing and dependency-aware scheduling for data-parallel clusters. In Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 81–97.
- [37] Evan Greensmith, Peter L Bartlett, and Jonathan Baxter. 2004. Variance reduction techniques for gradient estimates in reinforcement learning. *Journal of Machine Learning Research* 5, Nov (2004), 1471–1530.
- [38] Jayesh K Gupta, Maxim Egorov, and Mykel Kochenderfer. 2017. Cooperative multi-agent control using deep reinforcement learning. In Proceedings of the 2017 International Conference on Autonomous Agents and Multiagent Systems (AAMAS). 66–83.
- [39] Martin T Hagan, Howard B Demuth, Mark H Beale, and Orlando De Jesús. 1996. *Neural network design*. PWS publishing company Boston.
- [40] W Keith Hastings. 1970. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika* 1 (1970).
- [41] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D Joseph, Randy H Katz, Scott Shenker, and Ion Stoica. 2011. Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center. In Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation (NSDI).
- [42] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. 2007. Dryad: Distributed Data-parallel Programs from Sequential Building Blocks. In Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys). 59–72. <https://doi.org/10.1145/1272996.1273005>
- [43] Michael Isard, Vijayan Prabhakaran, Jon Currey, Udi Wieder, Kunal Talwar, and Andrew Goldberg. 2009. Quincy: fair scheduling for distributed computing clusters. In Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP). 261–276. <https://doi.org/10.1145/1629575.1629601>
- [44] James E. Kelley Jr and Morgan R. Walker. 1959. Critical-path planning and scheduling. In Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference (EJCC). 160–173.
- [45] Diederik P Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. Proceedings of the 7th International Conference on Learning Representations (ICLR) (2015).
- [46] Thomas N. Kipf and Max Welling. 2016. Semi-Supervised Classification with Graph Convolutional Networks. arXiv preprint arXiv:1609.02907 (2016). <http://arxiv.org/abs/1609.02907>
- [47] Prasad A. Kulkarni. 2011. JIT compilation policy for modern machines. In ACM SIGPLAN Notices, Vol. 46. 773–788.
- [48] Tom Leighton, Bruce Maggs, and Satish Rao. 1988. Universal packet routing algorithms. In Proceedings of the 29th annual Symposium on Foundations of Computer Science (FOCS). 256–269.
- [49] Zhuwen Li, Qifeng Chen, and Vladlen Koltun. 2018. Combinatorial optimization with graph convolutional networks and guided tree search. In Proceedings of the 32nd Conference on Neural Information Processing Systems (NeurIPS). 539–548.
- [50] Eric Liang and Richard Liaw. 2018. Scaling Multi-Agent Reinforcement Learning. <https://bair.berkeley.edu/blog/2018/12/12/rllib/>. (2018).
- [51] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971 (2015).
- [52] Chengzhi Lu, Kejiang Ye, Guoyao Xu, Cheng-Zhong Xu, and Tongxin Bai. 2017. Imbalance in the cloud: An analysis on alibaba cluster trace. In Proceedings of the 2017 IEEE International Conference on Big Data (BigData). IEEE, 2884–2892.
- [53] Hongzi Mao, Mohammad Alizadeh, Isha Menache, and Srikanth Kandula. 2016. Resource Management with Deep Reinforcement Learning. In Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets).
- [54] Hongzi Mao, Shannon Chen, Drew Dimmery, Shaun Singh, Drew Blaisdell, Yuandong Tian, Mohammad Alizadeh, and Eytan Bakshy. 2019. Real-world Video Adaptation with Reinforcement Learning. In Proceedings of the 2019 Reinforcement Learning for Real Life Workshop.
- [55] Hongzi Mao, Shaileshh Bojja Venkatakrishnan, Malte Schwarzkopf, and Mohammad Alizadeh. 2019. Variance Reduction for Reinforcement Learning in Input-Driven Environments. Proceedings of the 7th International Conference on Learning Representations (ICLR) (2019).
- [56] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: A Learned Query Optimizer. arXiv preprint arXiv:1904.03711 (2019).
- [57] Monaldo Mastrolilli and Ola Svensson. 2008. (Acyclic) job shops are hard to approximate. In Proceedings of the 49th IEEE Symposium on Foundations of Computer Science (FOCS). 583–592.
- [58] Isha Menache, Shie Mannor, and Nahum Shimkin. 2005. Basis function adaptation in temporal difference reinforcement learning. *Annals of Operations Research* 134, 1 (2005), 215–238.
- [59] Azalia Mirhoseini, Anna Goldie, Hieu Pham, Benoit Steiner, Quoc V Le, and Jeff Dean. 2018. A Hierarchical Model for Device Placement. In Proceedings of the 6th International Conference on Learning Representations (ICLR).
- [60] Azalia Mirhoseini, Hieu Pham, Quoc V Le, Benoit Steiner, Rasmus Larsen, Yuefeng Zhou, Naveen Kumar, Mohammad Norouzi, Samy Bengio, and Jeff Dean. 2017. Device Placement Optimization with Reinforcement Learning. In Proceedings of the 33rd International Conference on Machine Learning (ICML).
- [61] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, Demis Hassabis, Ioannis Antonoglou, Daan Wierstra, and Martin A. Riedmiller. 2015. Human-level control through deep reinforcement learning. *Nature* 518 (2015), 529–533.
- [62] Thanh Thi Nguyen, Ngoc Duy Nguyen, and Saeid Nahavandi. 2018. Deep Reinforcement Learning for Multi-Agent Systems: A Review of Challenges, Solutions and Applications. arXiv preprint arXiv:1812.11794 (2018).
- [63] Kay Ousterhout, Ryan Rasti, Sylvia Ratnasamy, Scott Shenker, and Byung-Gon Chun. 2015. Making Sense of Performance in Data Analytics Frameworks. In Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI). 293–307. <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/ousterhout>
- [64] Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta. 2017. Robust Adversarial Reinforcement Learning. In Proceedings of the 34th International Conference on Machine Learning (ICML). 2817–2826.
- [65] Chandrasekharan Rajendran. 1994. A no-wait flowshop scheduling heuristic to minimize makespan. *Journal of the Operational Research Society* 45, 4 (1994), 472–478.
- [66] Jeff Rasley, Konstantinos Karanasos, Srikanth Kandula, Rodrigo Fonseca, Milan Vojnovic, and Sriram Rao. 2016. Efficient Queue Management for Cluster Scheduling. In Proceedings of the 11th European Conference on Computer Systems (EuroSys). Article 36, 15 pages. <https://doi.org/10.1145/2901318.2901354>
- [67] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. 2015. Trust region policy optimization. In Proceedings of the 32nd International Conference on Machine Learning (ICML). 1889–1897.
- [68] Malte Schwarzkopf, Andy Konwinski, Michael Abd-El-Malek, and John Wilkes. 2013. Omega: flexible, scalable schedulers for large compute clusters. In Proceedings of the 8th ACM European Conference on Computer Systems (EuroSys). 351–364.
- [69] David B Shmoys, Clifford Stein, and Joel Wein. 1994. Improved approximation algorithms for shop scheduling problems. *SIAM J. Comput.* 23, 3 (1994), 617–632.
- [70] David Silver, Aja Huang, Christopher J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. 2016. Mastering the game of Go with deep neural networks and tree search. *Nature* 529 (2016), 484–503.
- [71] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. 2017. Mastering the game of Go without human knowledge. *Nature* 550, 7676 (2017), 354.
- [72] Richard. S. Sutton and Andrew. G. Barto. 2017. *Reinforcement Learning: An Introduction*, Second Edition. MIT Press.
- [73] TPC-H 2018. The TPC-H Benchmarks. www.tpc.org/tpch/. (2018).

- [74] Alexey Tumanov, Timothy Zhu, Jun Woo Park, Michael A. Kozuch, Mor Harchol-Balter, and Gregory R. Ganger. 2016. TetriSched: Global Rescheduling with Adaptive Plan-ahead in Dynamic Heterogeneous Clusters. In Proceedings of the 11th European Conference on Computer Systems (EuroSys). Article 35, 16 pages. <https://doi.org/10.1145/2901318.2901355>
- [75] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, Bikas Saha, Carlo Curino, Owen O'Malley, Sanjay Radia, Benjamin Reed, and Eric Baldeschwieler. 2013. Apache Hadoop YARN: Yet Another Resource Negotiator. In Proceedings of the 4th annual Symposium on Cloud Computing (SoCC). Article 5, 16 pages. <https://doi.org/10.1145/2523616.2523633>
- [76] Abhishek Verma, Madhukar Korupolu, and John Wilkes. 2014. Evaluating job packing in warehouse-scale computing. In Proceedings of the 2014 IEEE International Conference on Cluster Computing (CLUSTER). 48–56.
- [77] Abhishek Verma, Luis Pedrosa, Madhukar R. Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale cluster management at Google with Borg. In Proceedings of the 10th European Conference on Computer Systems (EuroSys). Bordeaux, France.
- [78] Lex Weaver and Nigel Tao. 2001. The optimal reward baseline for gradient-based reinforcement learning. In Proceedings of the 17th Conference on Uncertainty in Artificial Intelligence (UAI). 538–545.
- [79] Ronald J Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine learning 8, 3-4 (1992), 229–256.
- [80] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-tolerant Abstraction for In-memory Cluster Computing. In Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation (NSDI). 15–28. <http://dl.acm.org/citation.cfm?id=2228298.2228301>

附录

附录是未经同行评审的支持材料。

A 感知依赖关系的调度示例

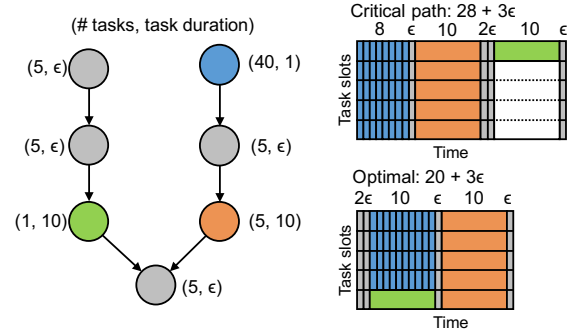


图16:最优的DAG感知调度会提前规划,并行执行蓝色和绿色阶段,使橙色和绿色阶段同时结束,底部的汇合阶段因此能立即运行。直接采用关键路径启发式方法则会聚焦右侧分支,使作业执行时间增加29%。图中术语: Critical path为关键路径调度; Optimal为最优调度; # tasks, task duration为任务数、任务时长; Task slots为任务槽位; Time为时间。

存在依赖关系的算子或“阶段”组成的有向无环图,在并行数据处理应用中很常见。

图16给出常见示例:两个分支在一个汇合阶段会合的DAG。简单的关键路径启发式方法会选择处理右侧分支,因为它包含更大的聚合作量:90任务秒,而左侧只有10任务秒。

但采用这一选择后,橙色阶段完成时,最终汇合阶段仍无法运行,因为其另一个父阶段,即绿色阶段,尚未完成。

像关键路径调度那样,接着完成绿色阶段,再运行汇合阶段,会使总完工时间为28+3ε。

相反,最优调度以20+3ε的时间完成该DAG,快29%。¹⁰

直观而言,理想调度应分配资源,使两个分支同时到达最终汇合阶段,随后无阻塞地执行。

B 强化学习背景

我们简要回顾本文使用的强化学习技术;详细综述及严格推导可参见Sutton和Barto的教材[72]。

强化学习。考虑图17的一般场景,强化学习智能体与环境交互。每一步 k ,智能体观察状态 s_k 并采取动作 a_k ;动作之后,环境状态转移到 s_{k+1} ,智能体收到奖励 r_k 作为反馈。状态转移和奖励具有随机性,并假定构成Markov过程:下一状态 s_{k+1} 和奖励 r_k 只依赖第 k 步的状态 s_k 与动作 a_k ,即给定它们后与过去条件独立。

在一般强化学习场景中,智能体只控制自己的动作,事先不知道状态转移概率或奖励函数;但它可以在训练中通过与环境交互,学习这些量。

强化学习训练按回合进行,每个回合包含一系列(状态、动作、奖励)观察,即每一步 $k \in [0, 1, \dots, T]$ 的 (s_k, a_k, r_k) ,其中 T 为回合长度。

为便于理解,我们先介绍最大化总奖励的强化学习形式: $\mathbb{E}[\sum_{k=0}^T r_k]$ 。但对调度问题而言,平均奖励形式(第5.3节)更合适。下文介绍怎样修改奖励信号,将目标转换为平均奖励。

¹⁰译者注:正文此处“快29%”以较慢方案为基准;图题中的“增加29%”应使用较快方案为基准,二者百分比不能直接等同。原文图题、正文数字和时间表达式均保留。

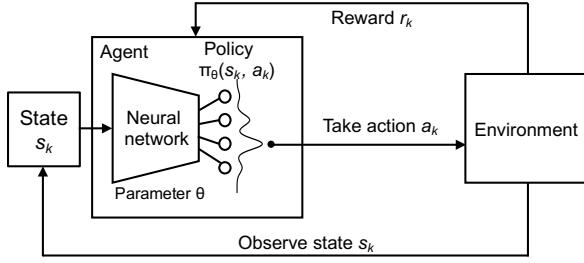


图 17: 使用神经网络的强化学习环境 [53, 72]。策略用神经网络参数化, 通过与环境反复交互、观察状态和采取动作, 进行迭代训练。图中术语: Agent 为智能体; State 为状态; Neural network 为神经网络; Policy 为策略; Environment 为环境; Take action 为采取动作; Observe state 为观察状态; Reward 为奖励; Parameter 为参数。

策略。智能体依据策略 $\pi(s_k, a_k)$ 选择动作, 该策略定义为状态 s_k 下采取动作 a_k 的概率。多数实际问题的候选 {状态、动作} 对太多, 无法用查找表保存策略。因此, 通常使用函数逼近器 [15, 58], 以数量可控的可调参数 θ 将策略表示为 $\pi_\theta(s_k, a_k)$ 。

多种形式的函数逼近器都可用于表示策略。常见选择包括状态/动作空间特征的线性组合, 即 $\pi_\theta(s_k, a_k) = \theta^T \phi(s_k, a_k)$; 以及近期使用神经网络 [39] 解决大规模强化学习任务 [61, 71]。

神经网络的优点, 是不需要人工构造特征, 并且训练时端到端可微。

策略梯度方法。我们关注通过对策略参数执行梯度下降来训练的一类强化学习算法。目标是最大化期望总奖励, 其梯度为:

$$\nabla_\theta \mathbb{E}_{\pi_\theta} \left[\sum_{k=0}^T r_k \right] = \mathbb{E}_{\pi_\theta} \left[\sum_{k=0}^T \nabla_\theta \log \pi_\theta(s_k, a_k) Q^{\pi_\theta}(s_k, a_k) \right], \quad (4)$$

其中, $Q^{\pi_\theta}(s_k, a_k)$ 表示在状态 s_k 下确定性地选择动作 a_k , 随后遵循策略 π_θ 时的期望总折扣奖励 [72, §13.2]。策略梯度方法的关键, 是利用当前策略生成的执行轨迹估计梯度。根据蒙特卡洛方法 [40], 智能体采样多条轨迹, 使用经验总折扣奖励 v_k 作为 $Q^{\pi_\theta}(s_k, a_k)$ 的无偏估计, 再通过梯度下降更新策略参数:

$$\theta \leftarrow \theta + \alpha \sum_{k=0}^T \nabla_\theta \log \pi_\theta(s_k, a_k) v_k, \quad (5)$$

其中, α 是学习率。该公式得到 REINFORCE 算法 [79]。其直观含义是, 方向 $\nabla_\theta \log \pi_\theta(s_k, a_k)$ 指明应怎样改变策略参数, 增大 $\pi_\theta(s_k, a_k)$, 即状态 s_k 下动作 a_k 的概率。式 5 沿这一方向迈出一步, 步长取决于回报 v_k 的大小。净作用是强化那些在经验上产生更好回报的动作。附录 C 介绍如何实现这种训练方法。

相比另一类基于价值的方法, 策略梯度更适合本调度场景, 原因有二。首先, 在无法确定所用神经网络架构的表达能力是否足够时, 基于策略的方法更容易设计。原因是, 基于价值的方法试图寻找 Bellman 方程的不动点 [13]。如果底层神经网络无法表达最优价值函数, 算法就可能难以收敛, 因为它试图收敛到网络无法表达的不动点。基于策略的方法没有这个问题: 无论策略网络的表达能力如何, 策略梯度算法都会在网络能够表达的策略空间内, 优化奖励目标。其次, 策略梯度方法允许使用依赖输入的基线 [55], 降低训练方差, 即第 5.3 节的挑战 #2。目前尚不清楚这一技术是否能够以及如何应用于基于价值的方法。

算法 1 用于训练 Decima 的策略梯度方法。

- 1: 对每次迭代执行
- 2: $\Delta\theta \leftarrow 0$
- 3: 采样回合长度 $\tau \sim \text{exponential}(\tau_{\text{mean}})$
- 4: 采样一个作业到达序列
- 5: 运行回合 $i = 1, \dots, N$:
 - $\{s_1^i, a_1^i, r_1^i, \dots, s_\tau^i, a_\tau^i, r_\tau^i\} \sim \pi_\theta$
- 6: 计算总奖励: $R_k^i = \sum_{k'=k}^\tau r_{k'}^i$
- 7: 对 $k = 1$ 至 τ 执行
- 8: 计算基线: $b_k = \frac{1}{N} \sum_{i=1}^N R_k^i$
- 9: 对 $i = 1$ 至 N 执行
- 10: $\Delta\theta \leftarrow \Delta\theta + \nabla_\theta \log \pi_\theta(s_k^i, a_k^i) (R_k^i - b_k)$
- 11: 结束循环
- 12: 结束循环
- 13: $\tau_{\text{mean}} \leftarrow \tau_{\text{mean}} + \epsilon$
- 14: $\theta \leftarrow \theta + \alpha \Delta\theta$
- 15: 结束循环

平均奖励形式。对调度问题而言, 最大化 $\lim_{T \rightarrow \infty} \mathbb{E}[1/T \sum_{k=0}^T r_k]$ 的平均奖励形式, 比前面讨论的总奖励更合适。

为将目标从奖励总和转换为平均奖励, 我们用差分奖励替代奖励 r_k 。具体而言, 每一步 k , 环境将智能体奖励修改为 $r_k \leftarrow r_k - \hat{r}$; 其中 $\hat{r}$ 是许多此前步骤、跨多个训练回合的奖励滑动平均值。

作这一修改后, 可以复用式 (4) 和式 (5) 中的同一策略梯度方法, 寻找最优策略。

Sutton 和 Barto [72, §10.3, §13.6] 给出该方法优化平均奖励目标的数学细节。

C 训练实现细节

算法 1 给出第 5.3 节所述 Decima 训练过程的伪代码。

具体而言, 第 3 行从指数分布采样回合长度 τ , 初始均值 τ_{mean} 较小, 使初始回合提早结束, 避免浪费训练时间; 见第 5.3 节的挑战 #1。然后, 第 4 行采样作业序列, 第 5 行使用它收集 N 个回合的经验。关键是第 8 行用同一作业序列计算基线 b_k , 降低到达过程随机性造成的方差; 见第 5.3 节的挑战 #2。第 10 行是式 (3) 所述的 REINFORCE 策略梯度算法。第 13 行增加平均回合长度, 即第 5.3 节挑战 #1 的课程学习过程。

最后, 第 14 行更新 Decima 的策略参数 θ 。

神经网络架构见第 6.1 节; Decima 训练的超参数如下。

到达作业数上限为 2000; 在整个训练期间, 回合终止概率从 5×10^{-7} 线性衰减至 5×10^{-8} 。

学习率 α 为 1×10^{-3} , 使用 Adam 优化器 [45] 进行梯度下降。对持续作业到达, 估计 $\hat{r}$ 的滑动窗口跨度为 10^5 个时间步; 见附录 B 的平均奖励形式。

最后, 全部实验中 Decima 都至少训练 50,000 次迭代。

我们用 TensorFlow [1] 实现 Decima 训练框架, 并使用 16 个工作进程, 在训练时并行计算采用相同作业序列的回合。

在配备 Intel Xeon E5-2640 CPU 和 Nvidia Tesla P100 GPU 的机器上, 每次训练迭代包括与仿真器交互、模型推理, 以及由全部训练工作进程更新模型, 约耗时 1.5 秒。

第 7 节的全部实验都使用训练中未见过的测试序列, 例如未见过的 TPC-H 作业组合、阿里巴巴生产轨迹的未见部分。

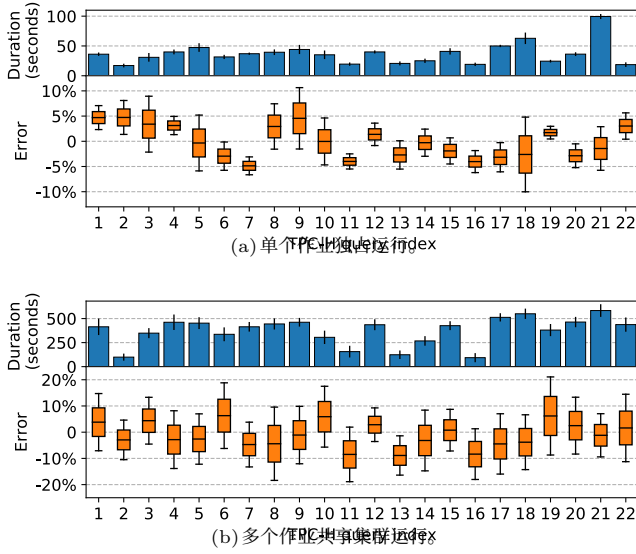


图 18: 用 Decima 作为调度智能体, 测试 Spark 仿真器的保真度。上部蓝色柱表示真实 Spark 作业时长的绝对值, 误差线为十次实验间的标准差; 下部橙色柱展示仿真误差的 95% 置信区间。对独占运行的单个作业, 仿真与实际作业时长的平均差异最多为 $\pm 5\%$; 对在集群中混合运行的全部 22 种查询, 平均误差最多为 $\pm 9\%$ 。图中术语: Duration (seconds) 为时长 (秒); Error 为误差; TPC-H query index 为 TPC-H 查询编号。

D 仿真器保真度

训练基础设施依赖能够忠实模拟集群中 Spark 作业执行的仿真器。

为验证保真度, 我们对 TPC-H 作业集合运行十次 (第 7.2 节), 测量仿真与真实 Spark 在作业完成时间上的差异, 同时考虑作业独占运行及与其他作业共享集群两种情况。

图 18 表明, 仿真器与各作业的实际运行时间高度吻合, 即使多个作业同时在集群运行也是如此。

具体而言, 作业独占运行时, 仿真平均误差位于真实运行时间的 5% 以内; 共享集群时位于 9% 以内。第 95th 百分位误差: 独占时 $\leq 10\%$, 共享时 $\leq 20\%$ 。

我们发现, 要达到这种准确性, 必须刻画 Spark 环境中的全部一阶影响 (第 6.2 节)。

例如, 不建模执行器在作业间迁移的延迟, 会使仿真运行时间持续低估真实情况。

在这种环境中训练, 将使策略过于积极地迁移执行器, 超出实际合理程度 (第 7.4 节)。

类似地, 忽略第一波与后续波次任务的差异, 或高并行度带来的减速开销, 会显著增大仿真运行时间的方差, 使 Decima 更难学习良好策略。

E Decima 状态表示的表达能力

只有状态表示、嵌入方案和神经网络架构能够表达强大的调度策略, Decima 才能学到它们 (第 7 节)。

式 (1) 结合两个非线性变换 $f(\cdot)$ 和 $g(\cdot)$, 使 Decima 能表达多种聚合函数。

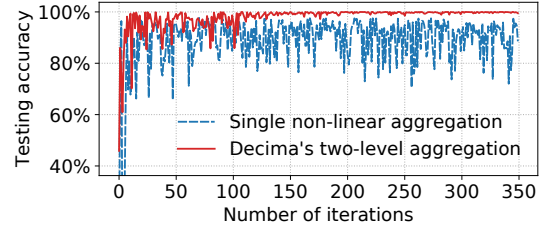


图 19: 通过监督学习训练后, Decima 的两层非线性变换能够表达计算关键路径所必需的最大值操作 (第 5.1 节); 因此, 相比标准图嵌入方法, 它在未见过的 DAG 上达到近乎完全准确的结果。图中术语: Number of iterations 为迭代次数; Testing accuracy 为测试准确率; Single non-linear aggregation 为单层非线性聚合; Decima's two-level aggregation 为 Decima 两层聚合。

例如, 当 $f \sim \log(\cdot/n)$ 、 $g \sim \exp(n \times \cdot)$, 且 $n \rightarrow \infty$ 时, 聚合操作计算子节点嵌入的最大值。¹¹

相反, 形如 $e_v = \sum_{u \in \xi(v)} f(e_u)$ 的标准聚合操作, 如果没有第二个非线性变换 $g(\cdot)$, 就不足以表达最大值操作。

因此, 这样的架构无法学会寻找图关键路径所需的最大值聚合。

在开发期间, 我们用一个简单合理性检查, 测试图嵌入方案的表达能力。

我们通过监督学习训练图神经网络, 使其输出大量随机图中各节点的关键路径值, 再检查它识别关键路径值最大节点的准确性。

图 19 展示 Decima 采用两层聚合的节点嵌入在未见过的图上的测试准确率, 并与只采用一个非线性变换的简单单层嵌入比较。

Decima 的节点嵌入成功学到最大值操作, 约 150 次迭代后即可准确识别关键路径; 标准嵌入无法表达关键路径, 因此始终无法达到稳定的高准确率。

F 多资源调度启发式方法的比较细节

评估 Decima 在多资源场景中的表现时 (第 7.3 节), 我们与多种启发式方法比较。

首先, 考虑第 7.2 节的最优调优加权公平启发式方法。

该方法根据作业总工作量, 赋予各作业一定执行器份额。

随后, 采用与单资源场景相同的方法选择阶段。

在可用执行器类型中, 先用尽最适合的类别, 再选择其他类别。

调度器确保跨不同执行器类型分配的资源总量, 不超过作业的加权公平份额。

其次, 与 Tetris 的资源装箱算法 [34] 比较。为最大化资源利用率, 我们针对每种执行器类型, 选择使请求资源向量与可用资源向量点积最大的 DAG 节点。

随后, 贪心地为该节点任务提供所需的最大并行度。

上述两种启发式方法分别缺少对方的关键调度因素, 即公平性和装箱; 两者都不理解 DAG 结构。

最后, 与 Graphene [36] 比较, 它的混合启发式方法结合了上述因素。

但我们的多资源环境具有离散执行器类别, 与原始 Graphene 所假定的连续、无限可分资源不同。

我们将 Graphene 改造成面向离散执行器的算法, 同时保留其核心: 以相同方式估计并分组“棘手”节点 [36, §4.1]。

¹¹ 译者注: 此处重复正文同一变换示例, 公式按原文保留; 关于内外变换顺序的疑点, 见正文相关译者注。

为确保棘手节点同时得到调度,我们动态压低某个DAG全部棘手节点的优先级,直到这些节点全部在可运行前沿中可用。

我们还按最优调优的加权公平划分启发式方法共享执行器,加入并行度控制;并优先使用最符合资源请求的执行器类型来实现资源装箱。

最后,对全部超参数进行网格搜索,例如选择棘手节点的阈值,使启发式方法在每个实验中达到最佳调度表现(第7.3节)。

G 多资源调度的进一步分析

第7.3节发现,Decima在多资源环境中处理持续到达作业时,平均JCT比当时先进的启发式方法低32%–43%。

Decima通过有选择地制造集群内存碎片获得这一效果:图12b表明,当借用大型执行器有助于尽快完成短作业并提高集群吞吐量时,Decima会选择这样做。

检查一次实验中的作业时长和执行器使用量时间序列,也能看到这一影响。

图20表明,无论是合成TPC-H工作负载,还是阿里巴巴轨迹,Decima在高负载时段都维持了更少的并发活动作业。

在繁忙时段,例如图20a1中快照50附近,Decima比最佳竞争启发式方法Graphene*更快地清理队列中的作业积压。

在这些时段,Decima为每个作业分配的执行器多于Graphene*,见图20a2和图20b2;例如,它有时为只需要小型执行器的作业借用大型执行器。

因此,在高集群负载下,Decima获得更低JCT和更高吞吐量,见图20a3和图20b3。

图21a和图21b详细比较Decima与Graphene*的执行器分配;图12b对应这一完整分析的第一列。

在x轴上,根据各作业总工作量、单位任务秒,对作业分箱。各图y轴表示Decima使用的执行器数,相对于Graphene*的执行器数归一化;大于1表示Decima使用更多执行器。

总体而言,Decima倾向于为每个作业分配比Graphene*更多的执行器。这有助于Decima更快完成作业,再转向其他作业,而不是同时推进很多作业,与第7.2节讨论的行为相似。

此外,Decima在小作业上使用更多大型执行器。这种积极分配大型执行器、浪费部分内存的方式,使繁忙时段的作业更快完成,见图20a3和图20b3;代价是留有部分未使用内存。

在资源碎片与优先完成小作业之间做平衡可能很繁琐,但Decima能通过与环境交互,自动学到强大的策略。

Decima在这里的部分优势,可能来自Graphene*被限制在离散执行器类别中。

如果集群允许任意、连续地为任务分配内存,那么只要CPU容量足够,就能把大执行器“槽位”拆成多个小执行器。

这种选择难以用Decima这样的有限动作空间表达;研究具有连续动作的强化学习能否用于集群调度,是有趣的未来方向。

H Decima的最优性

第7节表明,Decima在多种复杂集群环境中能够媲美或优于已有调度方案,包括真实Spark测试平台、真实集群轨迹仿真和多资源装箱环境。

不过,由于计算精确最优调度解[36,57]或紧下界在计算上不可处理,Decima在这些环境中的最优性仍然未知。¹²

尽管如此,为了解Decima与最优调度器的接近程度,我们可以在以暴力搜索不同调度的简化场景中测试它。

考虑第6.2节仿真的Spark调度框架,目标为一批作业的平均JCT。

¹²在本场景,即Spark基于执行器的调度中,我们发现基于总工作量或关键路径的下界太松,无法提供有意义的信息。

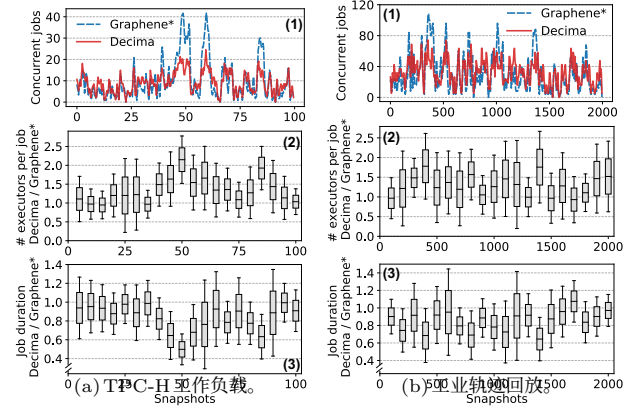


图20: 扩展的Spark多资源环境中,不同统计量的时间序列。比较Decima与最佳竞争启发式方法Graphene*。在繁忙时段,Decima为每个作业使用更多执行器,更快完成作业,并保持更少的并发作业。图中术语: Concurrent jobs为并发作业数; Snapshots为快照; # executors per job: Decima / Graphene为每作业执行器数量之比; Job duration: Decima / Graphene为作业时长之比; 算法名称保持原样。

为简化环境,我们关闭“波次”效应、执行器启动延迟及高并行度导致的任务减速效应。

因此,阶段时长与该阶段使用的执行器数严格成反比,即随并行资源线性扩展;调度器可以无开销地在作业间迁移执行器。

这一环境的主要挑战,是紧凑放置作业,并尽可能优先处理短作业。

为给一批 n 个作业寻找好的调度,我们穷举全部 $n!$ 个候选作业顺序,选择平均JCT最低的顺序。

为使穷举搜索可行,每批只考虑十个作业。对每种作业顺序,在每个调度事件(第5.2节)选择顺序中最靠前的未完成作业,再根据DAG关键路径决定作业内各阶段的完成顺序。

由于考虑全部候选作业顺序,该算法保证覆盖严格最短作业优先(SJF)等调度,后者能获得较小平均JCT。

我们认为该策略接近最优,因为经验上观察到作业顺序对TPC-H工作负载的平均JCT起主导作用(第7.4节)。

但穷举搜索还探索SJF的其他变体,例如优先处理能够利用并行性更快结束的作业,而不是优先处理总工作量更小、并行能力却更低的作业。

随后,在这一环境中训练未经修改的Decima智能体,设置类似第7.2节。

将该智能体与穷举搜索基线、最短作业优先关键路径启发式方法,以及调优的加权公平调度器比较;后两者见第7.2节。结果见图22。有三个关键观察。首先,与真实Spark集群中的结果不同(图9),SJF-CP优于调优的加权公平调度器。

这符合预期,因为SJF-CP严格优先处理小作业以最小化平均JCT;没有真实集群的复杂性时,这是一种好的策略。

其次,穷举搜索启发式方法优于SJF-CP。原因是, SJF-CP严格聚焦于先完成总工作量最小的作业,忽略DAG结构及其隐含的潜在并行性。

相反,穷举搜索会找到优先安排最快能执行完的作业的顺序,同时体现集群可用执行器、DAG结构和总工作量的影响。

搜索算法虽然不了解这些约束,但通过尝试不同作业顺序,找到了既正确排列作业、又利用集群资源尽快完成作业的调度。

第三,Decima的平均JCT与穷举搜索相当,甚至略优,平均改善9%。

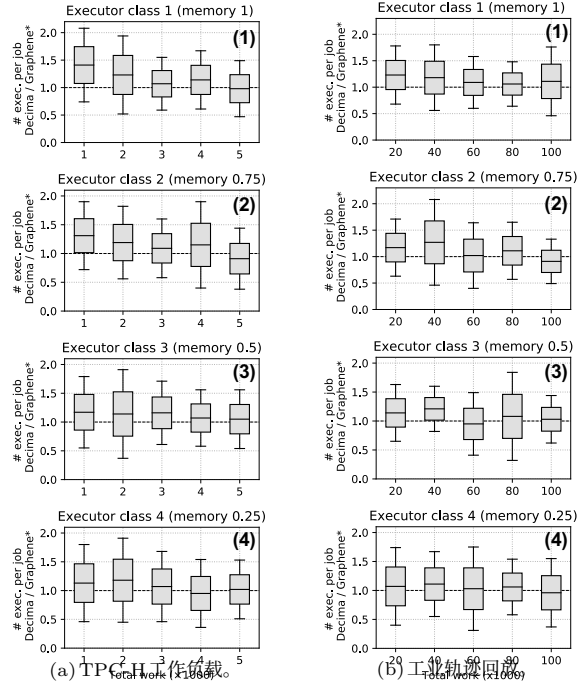


图21: 不同规模作业的执行器分配分析, 将Decima相对于Graphene*的分配归一化; 大于1表示Decima执行器更多, 小于1表示Graphene*更多。Decima倾向于分配更多执行器。
图中术语: Executor class 为执行器类别; memory 为内存; Total work 为总工作量; # exec. per job: Decima / Graphene* 为两种策略每作业执行器数量之比。

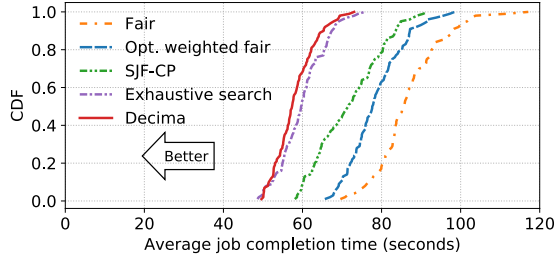


图22: 在简化调度环境中, 将Decima与近最优启发式方法比较。
图中术语: CDF 为累积分布函数; Average job completion time (seconds) 为平均作业完成时间 (秒); Better 的箭头指向较好结果; Fair 为公平; Opt. weighted fair 为最优加权公平; Exhaustive search 为穷举搜索; 其余算法名称保持原样。

我们发现, Decima更善于根据运行时当前结构动态调整作业优先级, 例如各依赖路径还剩多少工作量; 穷举搜索启发式方法则严格遵循离线静态搜索确定的顺序, 只控制作业何时启动。实验表明, Decima能自动学习达到离线最优作业顺序同等表现的调度算法。

I 将 Decima 泛化到不同环境

真实集群工作负载随时间变化, 可用机器也可能变化。

理想情况下, Decima应能把在特定负载和集群规模上训练的模型, 泛化到参数不同的相似工作负载。

Decima 训练场景	平均JCT (秒)
Decima, 按测试设置训练	3,290±680
Decima, 训练作业数缩小15×	3,540±450
Decima, 按测试设置训练	610±90
Decima, 训练执行器数缩小10×	630±70

表3: 工作负载或集群与训练设置不同的部署场景中, Decima 泛化良好。测试设置具有150个作业和10,000个执行器。

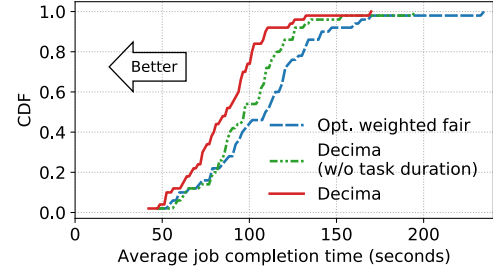


图23: 没有任务时长估计时, Decima在未见过的作业上表现变差, 但仍优于最佳启发式方法。

图中术语: CDF 为累积分布函数; Average job completion time (seconds) 为平均作业完成时间 (秒); Better 的箭头指向较好结果; Opt. weighted fair 为最优加权公平; w/o task duration 为不使用任务时长。

为测试这一点, 我们在工业负载的缩小版本上训练Decima智能体: 相对测试设置, 并发作业数缩小15×, 执行器数缩小10×。

表3比较这一智能体, 与直接在真实负载和集群规模上训练的智能体的表现。

Decima对参数变化稳健: 在少15×作业上训练的智能体泛化到测试负载时, 平均JCT降低7%; 在小10×的集群上训练的智能体泛化时, 平均JCT降低3%。¹³

泛化到更大的集群较为稳健, 因为即使可用资源大幅增多, 策略仍能正确限制各作业并行度。

相反, 泛化到包含更多作业的负载更难, 因为小规模训练缺少复杂作业组合的经验。

J 信息不完整时的 Decima

真实集群中, Decima偶尔会遇到没有可靠任务时长估计的新作业。

与从根本上依赖性能画像的启发式方法, 例如基于总工作量的加权公平调度不同, Decima仍能使用剩余信息, 获得合理调度策略。

在与第7.2节相同的设置下, 图23表明, 没有任务时长信息训练得到的策略, 仍优于最佳启发式方法, 因为Decima仍能利用图结构及其他信息, 例如任务数与高效并行度之间的相关性。

¹³译者注: “降低”按原文保留, 但对应表中3540对3290、630对610的平均JCT实际较高。此处应核查为性能下降/JCT增加, 而非JCT降低。原表及正文百分比均未改动。